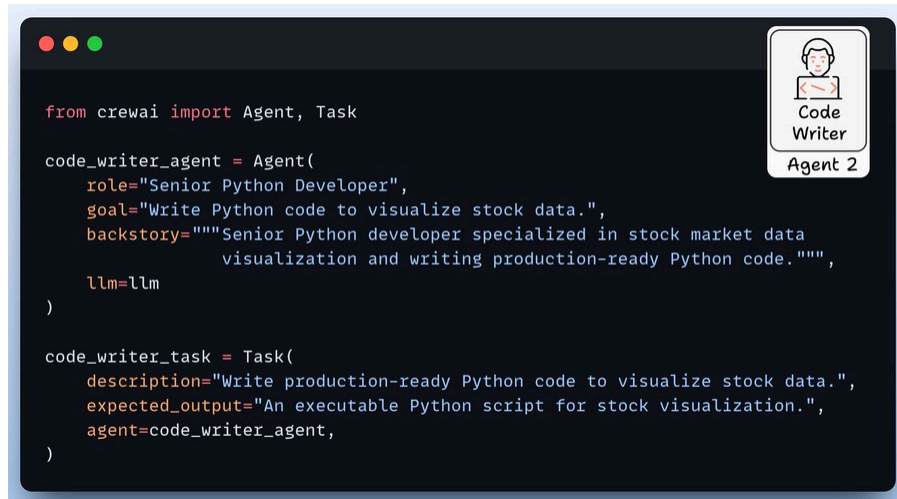


#3) Code Writer Agent

This agent writes Python code to visualize stock data using Pandas, Matplotlib, and Yahoo Finance libraries.



```
from crewai import Agent, Task

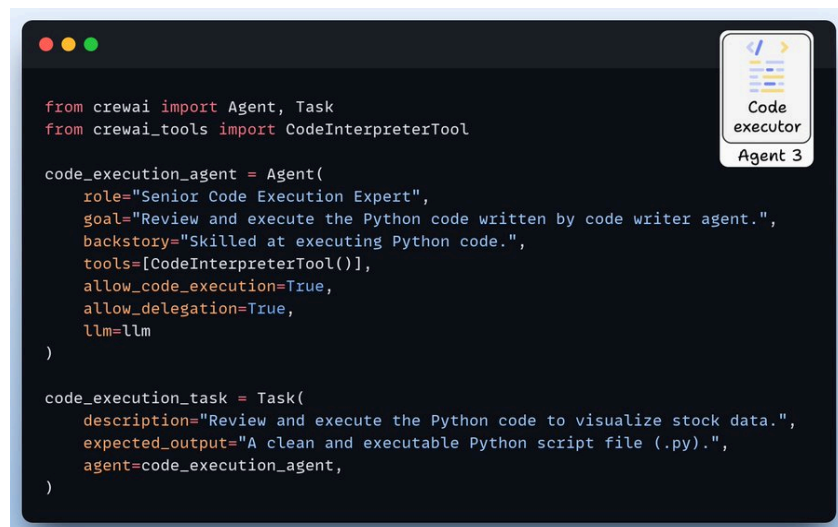
code_writer_agent = Agent(
    role="Senior Python Developer",
    goal="Write Python code to visualize stock data.",
    backstory="""Senior Python developer specialized in stock market data
        visualization and writing production-ready Python code.""",
    llm=llm
)

code_writer_task = Task(
    description="Write production-ready Python code to visualize stock data.",
    expected_output="An executable Python script for stock visualization.",
    agent=code_writer_agent,
)
```

#4) Code Executor Agent

This agent reviews and executes the generated Python code for stock data visualization.

It uses the code interpreter tool by CrewAI to execute the code in a secure sandbox environment.



```
from crewai import Agent, Task
from crewai_tools import CodeInterpreterTool

code_execution_agent = Agent(
    role="Senior Code Execution Expert",
    goal="Review and execute the Python code written by code writer agent.",
    backstory="Skilled at executing Python code.",
    tools=[CodeInterpreterTool()],
    allow_code_execution=True,
    allow_delegation=True,
    llm=llm
)

code_execution_task = Task(
    description="Review and execute the Python code to visualize stock data.",
    expected_output="A clean and executable Python script file (.py).",
    agent=code_execution_agent,
)
```

#5) Setup Crew and Kickoff

We set up and kick off our financial analysis crew to get the result shown below!



#6) Create MCP Server

Now, we encapsulate our financial analyst within an MCP tool and add two more tools to enhance the user experience.

- `save_code` -> Saves generated code to local directory
- `run_code_and_show_plot` -> Executes the code and generates a plot

```
server.py

from mcp.server.fastmcp import FastMCP
from finance_crew import run_financial_analysis

mcp = FastMCP("financial-analyst")

@mcp.tool()
def analyze_stock(query: str) -> str:
    """Analyzes stock market data based on query and generates
    executable Python code for analysis and visualization"""
    result = run_financial_analysis(query)
    return result

@mcp.tool()
def save_code(code: str) -> str:
    """Save the given code to a file stock_analysis.py"""
    with open('stock_analysis.py', 'w') as f:
        f.write(code)
    return "Code saved to stock_analysis.py"

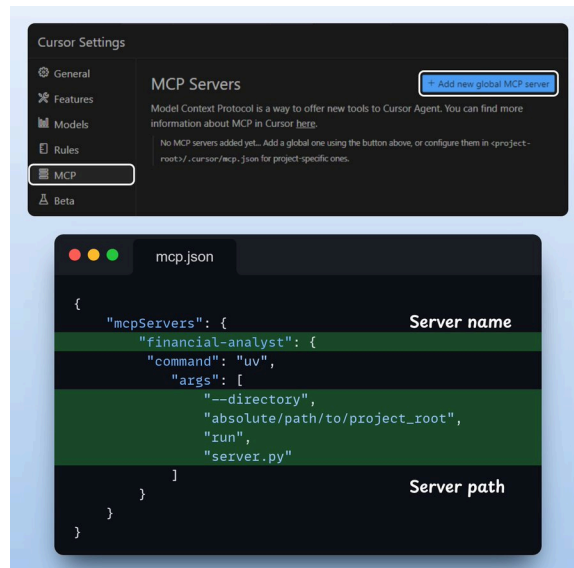
@mcp.tool()
def run_code_and_show_plot() -> str:
    """Execute code and generate a plot"""
    with open('stock_analysis.py', 'r') as f:
        exec(f.read())

if __name__ == "__main__":
    mcp.run(transport='stdio')
```

Model Context Protocol

#7) Integrate MCP server with Cursor

Go to: File → Preferences → Cursor Settings → MCP → Add new global MCP server. In the JSON file, add what's shown below



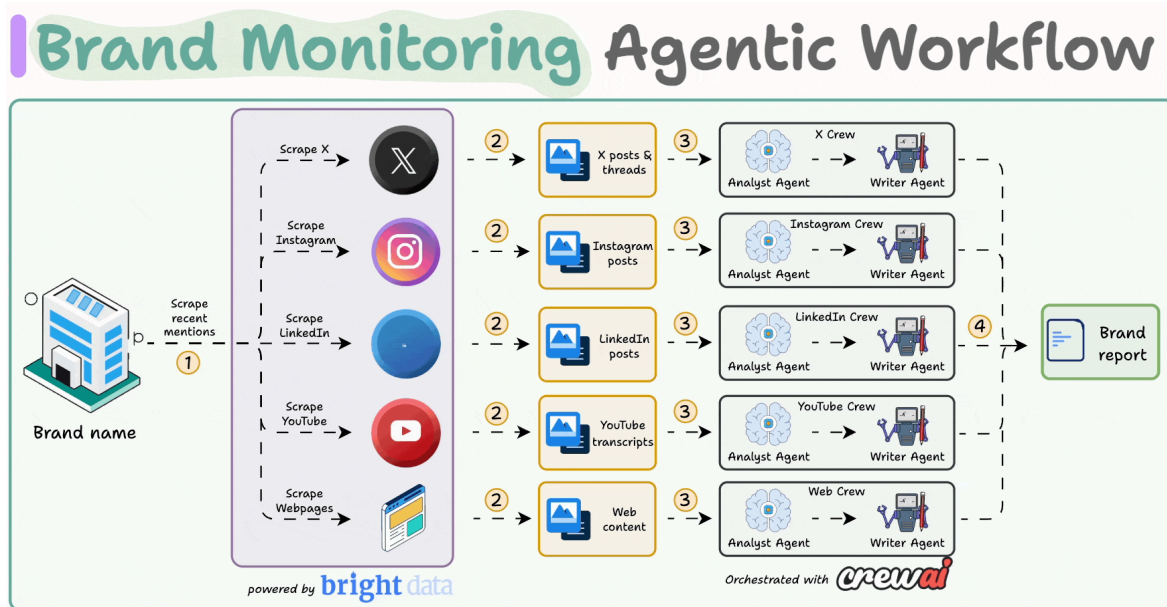
Done! Our financial analyst MCP server is live and connected to Cursor.



The code is available here:
<https://www.dailydoseofds.com/p/hands-on-building-an-mcp-powered-financial-analyst/>

#5) Brand Monitoring System

Build a multi agent brand monitoring app that scraps web mentions and produces insights about a company.



Tech stack:

- Bright Data to scrape data at scale
- CrewAI for orchestration
- Ollama to serve DeepSeek locally

Workflow:

- Use Bright Data to scrape brand mentions across X, Instagram, YouTube, websites, etc.
- Invoke platform-specific Crews to analyze the data and generate insights.
- Merge all insights to get the final report.

Let's implement this!

#1) Scraping tool

To monitor a brand, we must scrape data across various sources—X, YouTube, Instagram, websites, etc.

Thus, we'll first gather recent search results from Bright Data's SERP API.

See this code:

A screenshot of a code editor window titled 'book_writer.py'. The editor has a dark background with light blue and green accents. The code is written in Python and defines a web scraping tool. It imports 'BaseTool' from 'crewai.tools', 'BaseModel' and 'Field' from 'pydantic', and 'requests'. It then defines a 'BrightDataWebSearchToolInput' class as a Pydantic model with a 'title' field. Next, it defines a 'BrightDataWebSearchTool' class that inherits from 'BaseTool'. This class has attributes for 'name', 'description', and 'args_schema'. It also has a '_run' method that sets up a proxy using 'brd.superproxy.io', authenticates with a username and password, and sends a GET request to the Google SERP API. The response is returned as a JSON list of organic search results.

```
from crewai.tools import BaseTool
from pydantic import BaseModel, Field
import requests

class BrightDataWebSearchToolInput(BaseModel):
    """Input schema for BrightDataWebSearchTool."""
    title: str = Field(..., description="Topic of the book to write about.")

class BrightDataWebSearchTool(BaseTool):
    name: str = "Web Search Tool"
    description: str = "Tool to search Google and retrieve the results."
    args_schema: Type[BaseModel] = BrightDataWebSearchToolInput

    def _run(self, title):

        host = 'brd.superproxy.io'
        port = 33335

        username = '<Get from brightdata.com in SERP API>'
        password = '<Get from brightdata.com in SERP API>'

        proxies = {
            'http': f'http://{username}:{password}@{host}:{port}',
            'https': f'http://{username}:{password}@{host}:{port}'
        }

        url = f"https://www.google.com/search?q={title}&brd_json=1&num=500"
        response = requests.get(url, proxies=proxies, verify=False)

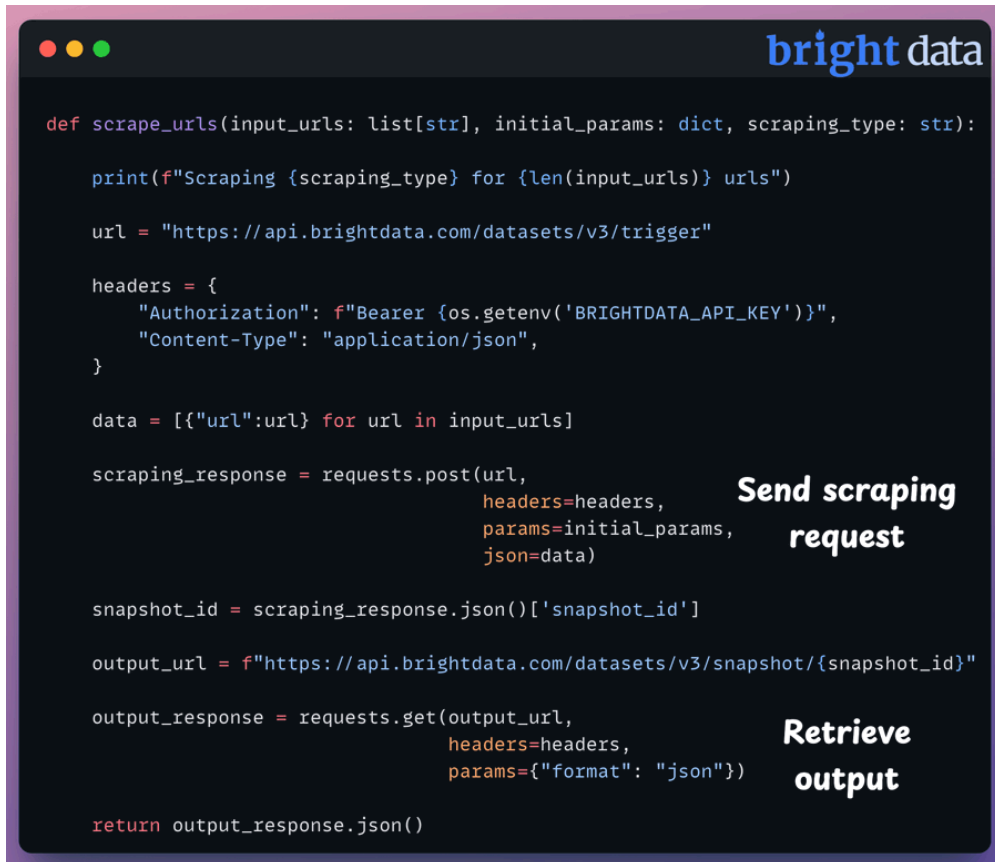
        return response.json()['organic']
```

#2) Platform-specific scraping function

The above output will contain links to web pages, X posts, YouTube videos, Instagram posts, etc.

To scrape those sources, we use Bright Data's platform-specific scrapers.

Check this code:



```
def scrape_urls(input_urls: list[str], initial_params: dict, scraping_type: str):  
    print(f"Scraping {scraping_type} for {len(input_urls)} urls")  
  
    url = "https://api.brightdata.com/datasets/v3/trigger"  
  
    headers = {  
        "Authorization": f"Bearer {os.getenv('BRIGHTDATA_API_KEY')}",  
        "Content-Type": "application/json",  
    }  
  
    data = [{"url":url} for url in input_urls]  
  
    scraping_response = requests.post(url,  
                                     headers=headers,  
                                     params=initial_params,  
                                     json=data)  
  
    snapshot_id = scraping_response.json()['snapshot_id']  
  
    output_url = f"https://api.brightdata.com/datasets/v3/snapshot/{snapshot_id}"  
  
    output_response = requests.get(output_url,  
                                   headers=headers,  
                                   params={"format": "json"})  
  
    return output_response.json()
```

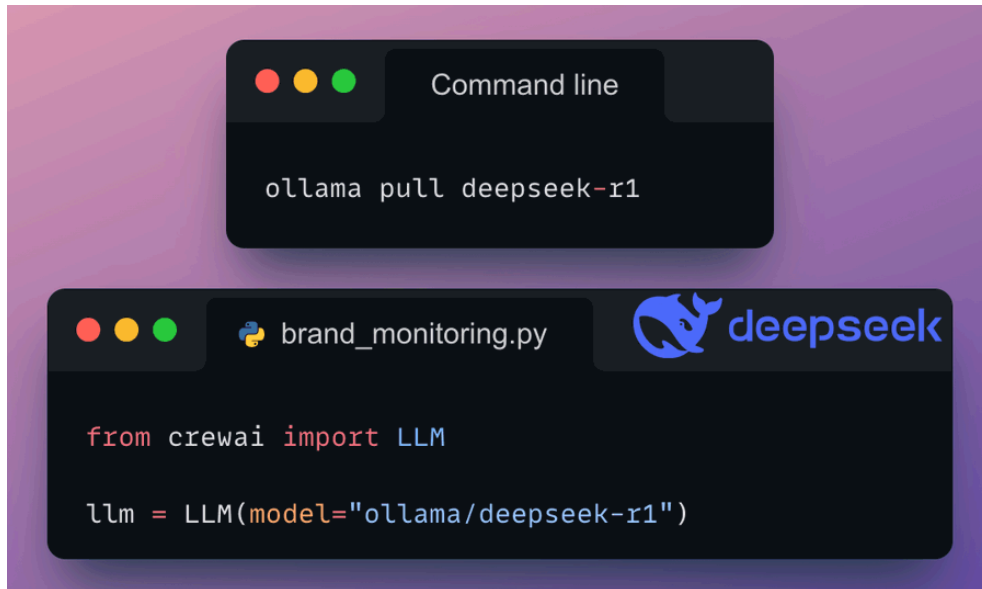
#3) Set up DeepSeek R1 locally

We'll serve R1 locally through Ollama.

To do this:

- First, we download it locally.
- Next, we define it with the CrewAI's LLM class.

Here's the code:



```
ollama pull deepseek-r1
```

```
from crewai import LLM

llm = LLM(model="ollama/deepseek-r1")
```

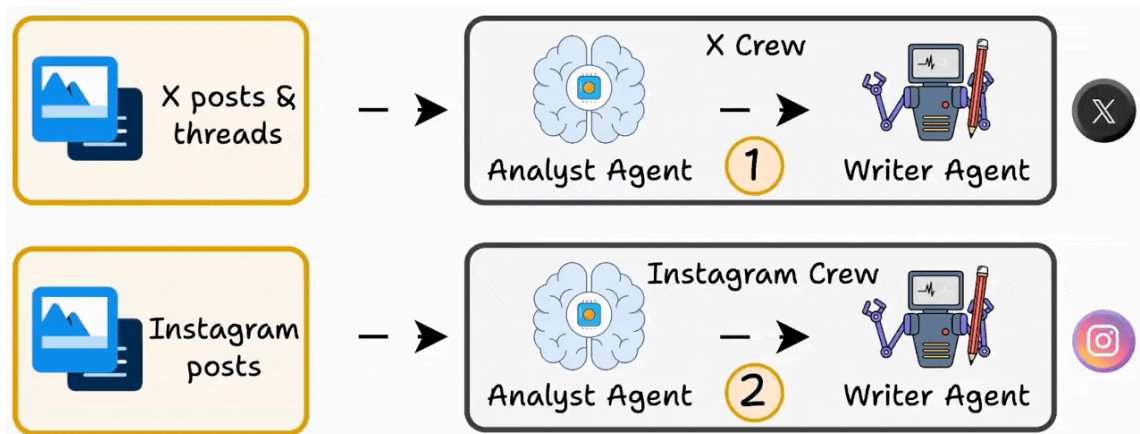
#4) Crew Setup

We will have multiple Crews, one for each platform (X, Instagram, YouTube, etc.)

Each Crew will have two Agents:

- Analysis Agent → It analyses the scraped content.
- Writer Agent → It produces insights from the analysis.

Below, let's implement the X Crew!



#5) X Analyst Agent

This Agent analyzes the posts scraped by Bright Data and extracts key insights. It is also assigned a task to do so.

Here's how it's done:



```
from crewai import Agent, Task

analysis_agent = Agent(role="X Analysis Agent"
                        goal="""Analyze the usage of {brand_name} in X posts
                        with details like URL, Views, Likes, Replies,
                        Reposts, Hashtags, Quotes, Bookmarks, Description,
                        Tagged Users, Poster ID"""
                        backstory="""You're an X post analysis expert with an
                        understanding of the platform and its algorithms.
                        You can analyze posts, description, views, likes,
                        replies, reposts, hashtags, quotes, bookmarks,
                        tagged users and analyse the usage of a brand
                        mentioned in the posts.""")

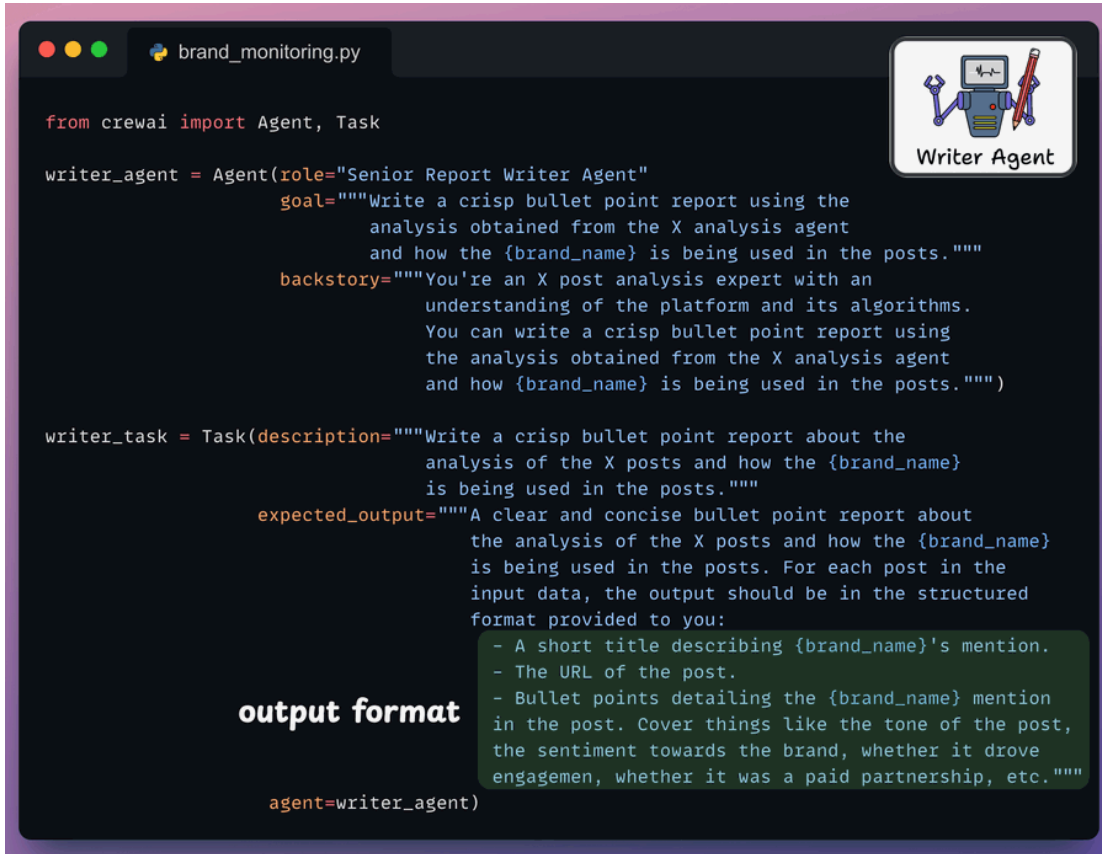
analysis_task = Task(description="""Analyze the usage of {brand_name} in X posts
                        with these details URL, Views, Likes, Replies,
                        Reposts, Hashtags, Quotes, Bookmarks, Description,
                        Tagged Users, Poster ID.

                        Scraped X posts
                        This is the list of scraped data:
                        {x_data}"""
                        expected_output="""A concise analysis of X posts and how
                        {brand_name} is being portrayed in them. You can
                        cover things like the tone of the post, the sentiment
                        towards the brand, whether it received engagement,
                        whether it was a paid partnership, etc.""")
                        agent=analysis_agent)
```

#6) X Writer Agent

The Agent takes the output of the X analyst agent and generates insights.

Here's the code:



```
from crewai import Agent, Task

writer_agent = Agent(role="Senior Report Writer Agent"
                     goal="""Write a crisp bullet point report using the
                           analysis obtained from the X analysis agent
                           and how the {brand_name} is being used in the posts.""",
                     backstory="""You're an X post analysis expert with an
                           understanding of the platform and its algorithms.
                           You can write a crisp bullet point report using
                           the analysis obtained from the X analysis agent
                           and how {brand_name} is being used in the posts.""")

writer_task = Task(description="""Write a crisp bullet point report about the
                           analysis of the X posts and how the {brand_name}
                           is being used in the posts.""",
                   expected_output="""A clear and concise bullet point report about
                           the analysis of the X posts and how the {brand_name}
                           is being used in the posts. For each post in the
                           input data, the output should be in the structured
                           format provided to you:
                           - A short title describing {brand_name}'s mention.
                           - The URL of the post.
                           - Bullet points detailing the {brand_name} mention
                           in the post. Cover things like the tone of the post,
                           the sentiment towards the brand, whether it drove
                           engagement, whether it was a paid partnership, etc.""",
                   agent=writer_agent)
```

output format

#7) Create a Flow

Finally, we use CrewAI Flows to orchestrate the workflow:

- We start the Flow by using the Scraping tool.
- Next, we invoke platform-specific scrapers.
- Finally, we invoke platform-specific Crews.

Check this out:

```
brand_monitoring.py

from crewai.flow import Flow, listen, start
from pydantic import BaseModel

class BrandMonitoringFlow(Flow[BrandMonitoringState]):

    @start()
    def scrape_data(self):
        search_tool = BrightDataWebSearchTool() Scrape URLs of brand mentions
        self.state.search_result = search_tool(self.state.brand_name)

    @listen(scrape_data)
    def scrape_platform_data_and_analyse(self):

        # X scraping
        X_posts = scrape_urls(self.state.search_response.x_urls,
                              X_params, "X")

        X_crew = XCrew().crew()
        X_reponse = X_crew.kickoff(inputs={"X_data":X_posts, Scrape X posts
                                          "brand_name": self.state.brand_name})

        # YouTube scraping
        YouTube_transcripts = scrape_urls(self.state.search_response.YouTube_urls,
                                           YouTube_params, "X") Scrape YouTube
                                                                    videos

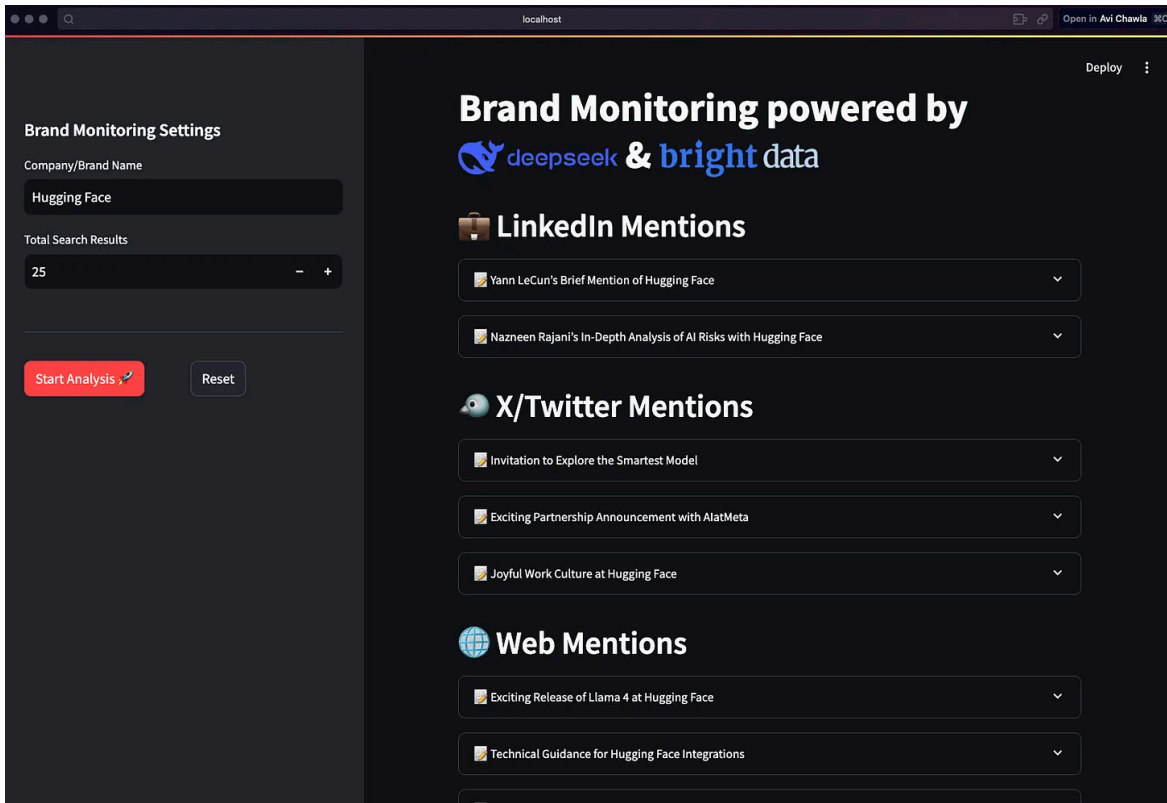
        YouTube_crew = YouTubeCrew().crew()
        YouTube_reponse = YouTube_crew.kickoff(inputs={"X_data":YouTube_posts,
                                                       "brand_name": self.state.brand_name})

        # repeat for all plaforms
```

#8) Streamlit UI and Kick off the Flow

Finally, we wrap the app in a clear Streamlit interface for interactivity and run the Flow.

Check the final outcome:



Done!

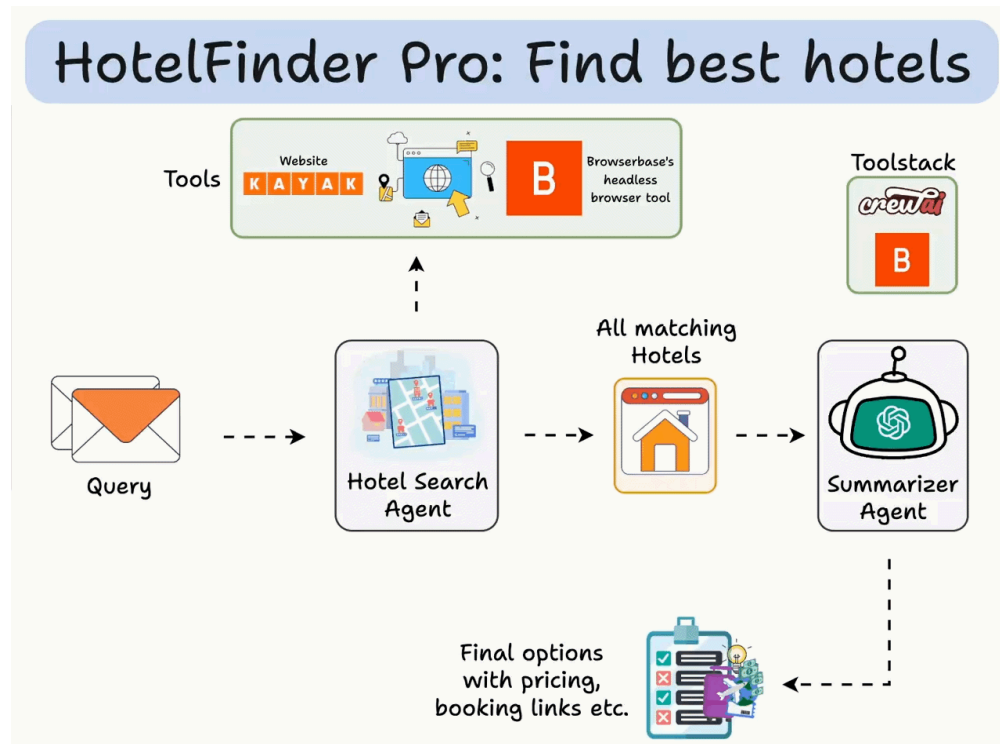
You're now ready to monitor any brand, track all mentions, and generate insights about a company.



The code is available here:
<https://www.dailydoseofds.com/p/hands-on-build-a-multi-agent-brand-monitoring-system/>

#6) Multi-agent Hotel Finder

Build an Agentic workflow that parses a travel query, fetches live flights and hotel data from Kayak, and summarizes the best options.



Tech stack:

- CrewAI for multi-agent orchestration
- Browserbase's headless browser tool
- Ollama to locally serve DeepSeek-R1

Workflow:

- Parse the query (location, dates etc.) to create a Kayak search URL
- Visit the URL and extract top 5 flights
- For each hotel, find pricing and more info
- Summarize hotel info

Let's implement this!

#1) Define LLM

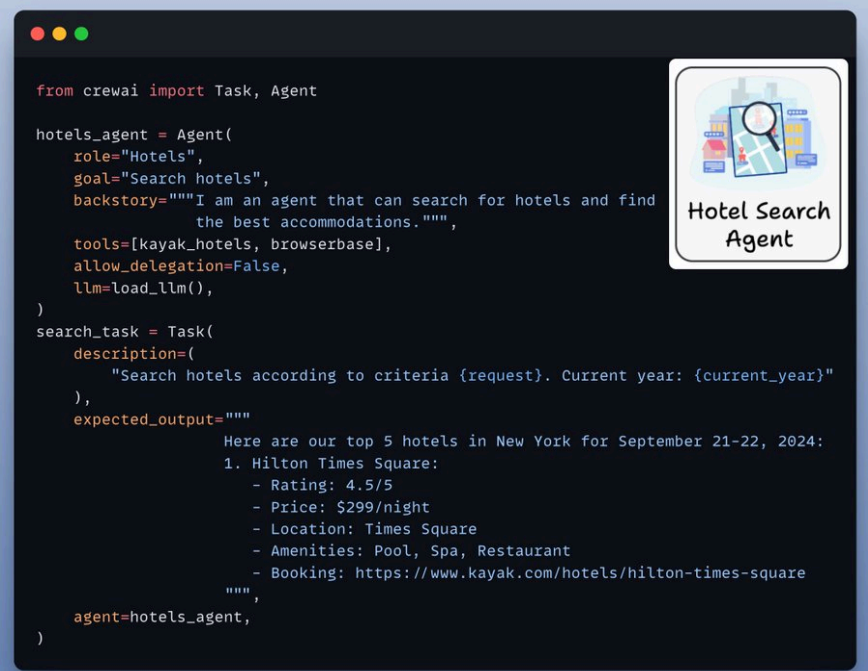
CrewAI nicely integrates with all the popular LLMs and providers out there!

Here's how you set up a local DeepSeek using Ollama:



#2) Hotel Search Agent

This agent mimics a real human searching for hotels by browsing the web. It's powered by browserbase's headless-browser tool and can look up hotels on sites like Kayak.



```
from crewai import Task, Agent

hotels_agent = Agent(
    role="Hotels",
    goal="Search hotels",
    backstory="""I am an agent that can search for hotels and find
                the best accommodations.""",
    tools=[kayak_hotels, browserbase],
    allow_delegation=False,
    llm=load_llm(),
)

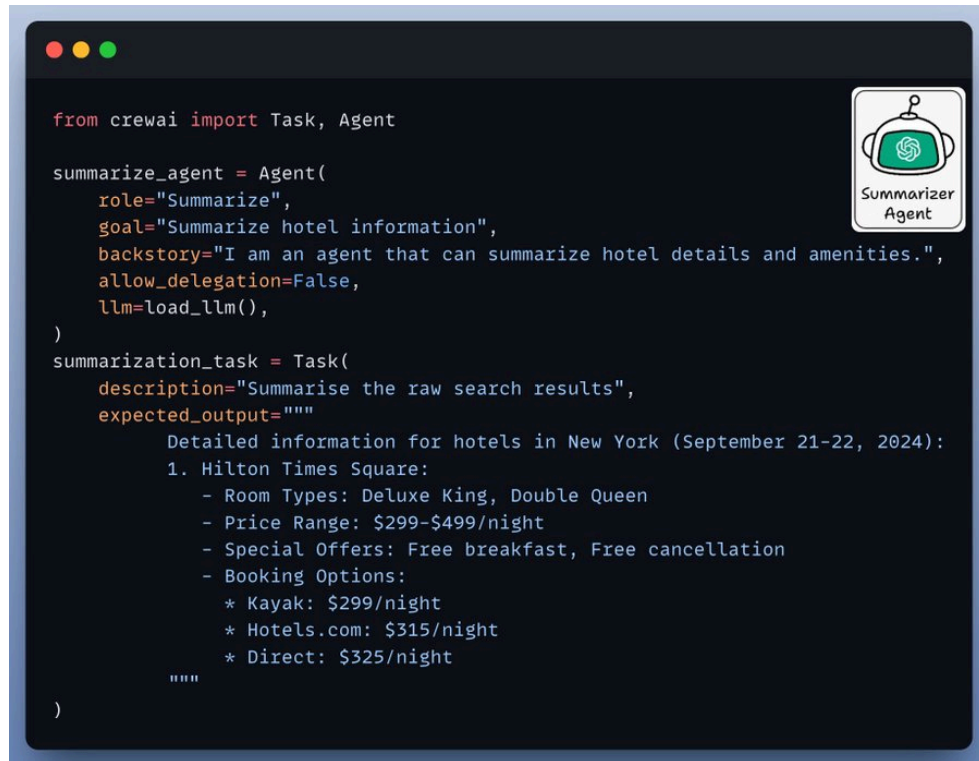
search_task = Task(
    description=(
        "Search hotels according to criteria {request}. Current year: {current_year}"
    ),
    expected_output="""
        Here are our top 5 hotels in New York for September 21-22, 2024:
        1. Hilton Times Square:
            - Rating: 4.5/5
            - Price: $299/night
            - Location: Times Square
            - Amenities: Pool, Spa, Restaurant
            - Booking: https://www.kayak.com/hotels/hilton-times-square
        """,
    agent=hotels_agent,
)
```

#3) Summarisation Agent

After retrieving the hotel details, we need a concise summary of all available options.

This is where our Summarization Agent steps in to make sense of the results for easy reading.

Check this out:



```
from crewai import Task, Agent

summarize_agent = Agent(
    role="Summarize",
    goal="Summarize hotel information",
    backstory="I am an agent that can summarize hotel details and amenities.",
    allow_delegation=False,
    llm=load_llm(),
)

summarization_task = Task(
    description="Summarise the raw search results",
    expected_output="""
        Detailed information for hotels in New York (September 21-22, 2024):
        1. Hilton Times Square:
            - Room Types: Deluxe King, Double Queen
            - Price Range: $299-$499/night
            - Special Offers: Free breakfast, Free cancellation
            - Booking Options:
              * Kayak: $299/night
              * Hotels.com: $315/night
              * Direct: $325/night
        """)
```

Now that we have both our agents ready, it's time to understand the tools powering them.

1. Kayak tool
2. Browserbase tool

Let's write their code one-by-one.

Tools



#4) Kayak tool

A custom Kayak tool to translate the user input into a valid Kayak search URL.

(FYI Kayak is a popular for hotel and flight booking)

```
from crewai.tools import tool
from typing import Optional

# --- New Hotel Search Function ---
@tool("Kayak Hotel Tool")
def kayak_hotel_search(
    location: str, check_in_date: str, check_out_date: str, num_adults: int = 2
) -> str:
    """
    Generates a Kayak URL for hotel searches based on location, dates, and number of adults.

    :param location_query: The location string used by Kayak (e.g., 'Hisar,Haryana,India-p15321')
    :param check_in_date: The check-in date in 'YYYY-MM-DD' format
    :param check_out_date: The check-out date in 'YYYY-MM-DD' format
    :param num_adults: The number of adults (defaults to 2)
    :return: The Kayak URL for the hotel search
    """
    URL = f"https://www.kayak.co.in/hotels/{location}/{check_in_date}/{check_out_date}"
    return URL

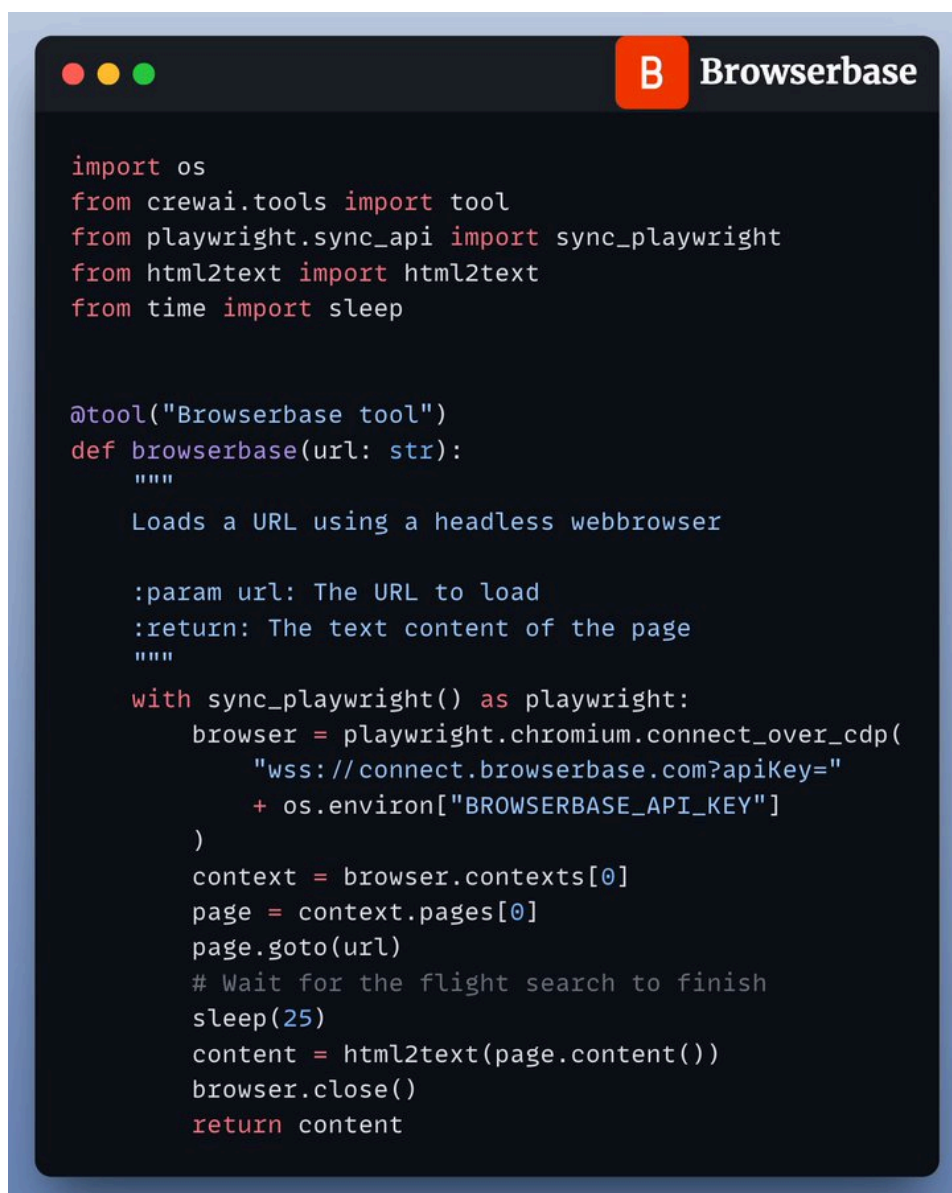
# Export the new decorated function
kayak_hotels = kayak_hotel_search
```

#5) Browserbase Tool

The hotel search agent uses the Browserbase tool to simulate human browsing and gather hotel data.

To be precise it automatically navigates the Kayak website and interacts with the web page.

Check this out:

A screenshot of a code editor window with a dark background and light-colored text. The window has a title bar with three colored circles (red, yellow, green) on the left and a title 'Browserbase' on the right. The code is written in Python and defines a tool for interacting with a headless web browser. It includes imports for 'os', 'crewai.tools', 'playwright.sync_api', 'html2text', and 'time'. The main function 'browserbase' is decorated with '@tool' and takes a 'url' parameter. It uses 'playwright' to connect to a browser, navigate to the specified URL, wait for 25 seconds, and return the page content. The code is as follows:

```
import os
from crewai.tools import tool
from playwright.sync_api import sync_playwright
from html2text import html2text
from time import sleep

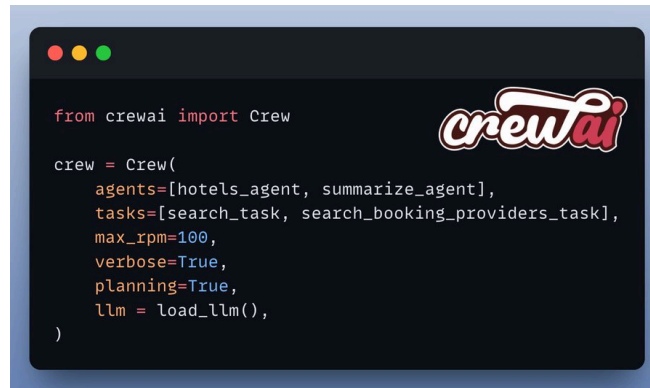
@tool("Browserbase tool")
def browserbase(url: str):
    """
    Loads a URL using a headless webbrowser

    :param url: The URL to load
    :return: The text content of the page
    """
    with sync_playwright() as playwright:
        browser = playwright.chromium.connect_over_cdp(
            "wss://connect.browserbase.com?apiKey="
            + os.environ["BROWSERBASE_API_KEY"]
        )
        context = browser.contexts[0]
        page = context.pages[0]
        page.goto(url)
        # Wait for the flight search to finish
        sleep(25)
        content = html2text(page.content())
        browser.close()
        return content
```

#6) Setup Crew

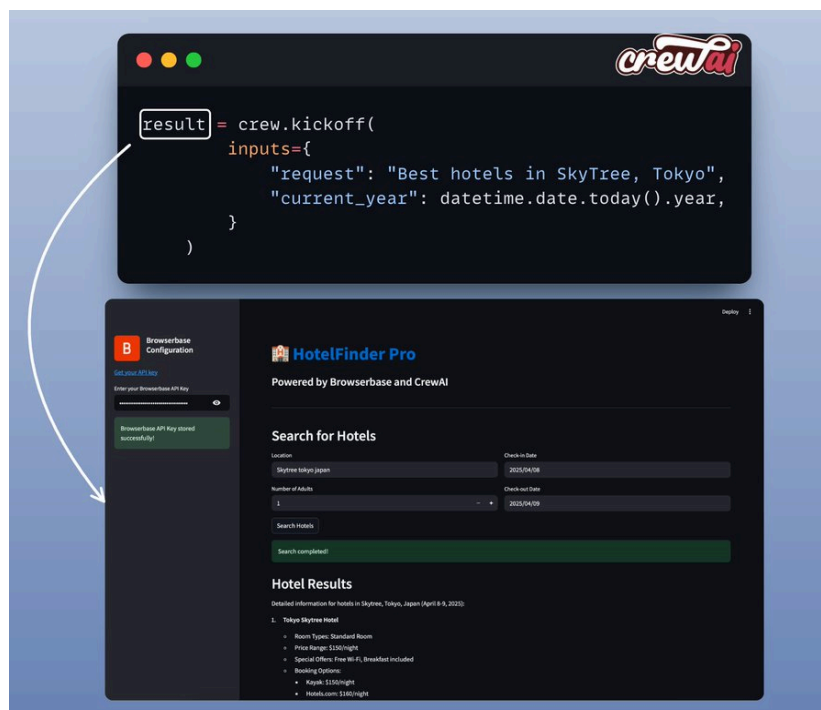
Once the agents and tools are defined, we orchestrate them using CrewAI.

Define their tasks, sequence their actions, and watch them collaborate in real time! Check this out:



#7) Kickoff and results

Finally, we feed the user's request (location, dates etc.) into the Crew and let it run! Check this out:



Streamlit UI

To make this accessible, we wrapped the entire system in a @Streamlit interface.

It's a simple chat-like UI where you enter your location and other details and see the results in real time!

Check this out:

Browserbase Configuration

Get your API key

Enter your Browserbase API Key

Browserbase API Key stored successfully!

HotelFinder Pro

Powered by Browserbase and CrewAI

Search for Hotels

Location: Skytree tokyo japan

Check-in Date: 2025/04/08

Number of Adults: 1

Check-out Date: 2025/04/09

Search Hotels

Search completed!

Hotel Results

Detailed information for hotels in Skytree, Tokyo, Japan (April 8-9, 2025):

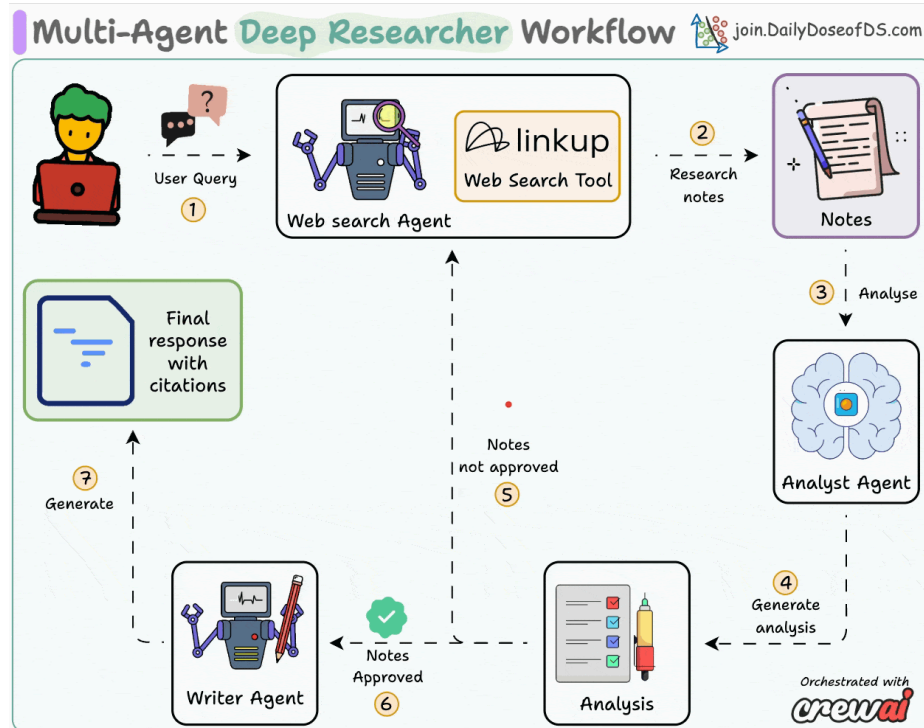
1. Tokyo Skytree Hotel
 - Room Types: Standard Room
 - Price Range: \$150/night
 - Special Offers: Free Wi-Fi, Breakfast included
 - Booking Options:
 - Kayak: \$150/night
 - Hotels.com: \$160/night



The code is available here:
<https://github.com/patchy631/ai-engineering-hub/tree/main/hotel-booking-crew>

#7) Multi-agent Deep Researcher

ChatGPT has a deep research feature. It helps you get detailed insights on any topic. Learn how you can build a 100% local alternative to it.



Tech stack:

- Linkup platform for deep web research
- CrewAI for multi-agent orchestration
- Ollama to locally serve DeepSeek
- Cursor as MCP host

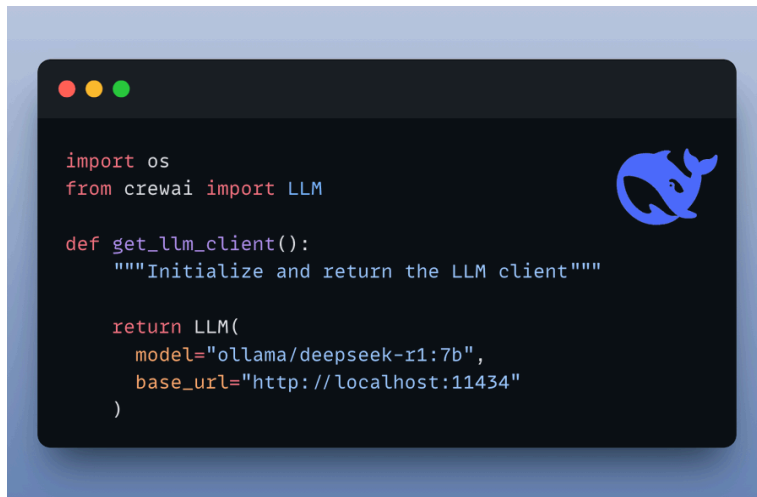
Workflow:

- User submits a query
- Web search agent runs deep web search via Linkup
- Research analyst verifies and deduplicates results
- Technical writer crafts a coherent response with citations

Let's implement this!

#1) Setup LLM

We'll use a locally served DeepSeek-R1 using Ollama.



#2) Define Web Search Tool

We'll use Linkup platform's powerful search capabilities, which rival Perplexity and OpenAI, to power our web search agent. This is done by defining a custom tool that our agent can use.

```
import os
from typing import Type
from pydantic import BaseModel, Field
from linkup import LinkupClient
from crewai.tools import BaseTool

class LinkUpSearchInput(BaseModel):
    """Input schema for LinkUp Search Tool."""
    query: str = Field(description="The search query to perform")
    depth: str = Field(default="standard", description="Depth of search: 'standard' or 'deep'")
    output_type: str = Field(
        default="searchResults",
        description="Output type: 'searchResults' or 'sourcedAnswer'"
    )

class LinkUpSearchTool(BaseTool):
    name: str = "LinkUp Search"
    description: str = "Retrieve info from the web using LinkUp and return results"
    args_schema: Type[BaseModel] = LinkUpSearchInput

    def _run(self, query: str, depth: str = "standard",
             output_type: str = "searchResults") -> str:
        """Execute LinkUp search and return results."""
        # Initialize LinkUp client with API key from environment variables
        linkup_client = LinkupClient(api_key=os.getenv("LINKUP_API_KEY"))

        # Perform search
        search_response = linkup_client.search(query=query,
                                              depth=depth,
                                              output_type=output_type)

        return str(search_response)
```



#3) Define Web Search Agent

The web search agent gathers up-to-date information from the internet based on user query. The linkup tool we defined earlier is used by this agent.

```
from crewai import Agent, Task

linkup_search_tool = LinkUpSearchTool()

client = get_llm_client()

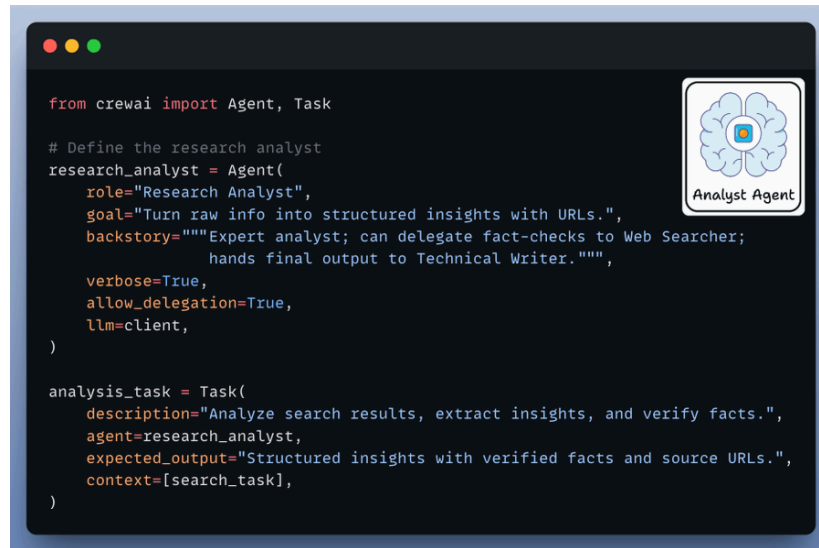
web_searcher = Agent(
    role="Web Searcher",
    goal="Retrieve relevant info with citations (source URLs)",
    backstory="Expert searcher; forwards results to Research Analyst only.",
    verbose=True,
    allow_delegation=True,
    tools=[linkup_search_tool],
)

search_task = Task(
    description=f"Search for comprehensive information about: {query}.",
    agent=web_searcher,
    expected_output="Detailed raw search results including sources (urls).",
    tools=[linkup_search_tool]
)
```



#4) Define Research Analyst Agent

This agent transforms raw web search results into structured insights, with source URLs. It can also delegate tasks back to the web search agent for verification and fact-checking.



```
from crewai import Agent, Task

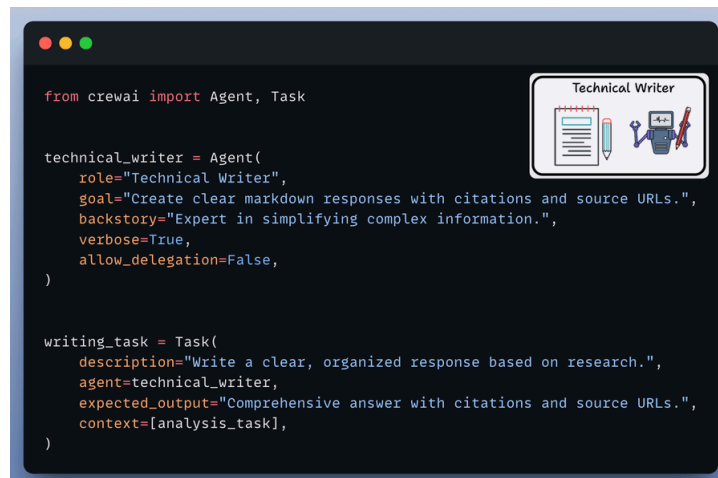
# Define the research analyst
research_analyst = Agent(
    role="Research Analyst",
    goal="Turn raw info into structured insights with URLs.",
    backstory="Expert analyst; can delegate fact-checks to Web Searcher; hands final output to Technical Writer.",
    verbose=True,
    allow_delegation=True,
    llm=client,
)

analysis_task = Task(
    description="Analyze search results, extract insights, and verify facts.",
    agent=research_analyst,
    expected_output="Structured insights with verified facts and source URLs.",
    context=[search_task],
)
```



#5) Define Technical Writer Agent

It takes the analyzed and verified results from the analyst agent and drafts a coherent response with citations for the end user.



```
from crewai import Agent, Task

technical_writer = Agent(
    role="Technical Writer",
    goal="Create clear markdown responses with citations and source URLs.",
    backstory="Expert in simplifying complex information.",
    verbose=True,
    allow_delegation=False,
)

writing_task = Task(
    description="Write a clear, organized response based on research.",
    agent=technical_writer,
    expected_output="Comprehensive answer with citations and source URLs.",
    context=[analysis_task],
)
```



#6) Setup Crew

Finally, once we have all the agents and tools defined we set up and kickoff our deep researcher crew.



#7) Create MCP Server

Now, we'll encapsulate our deep research team within an MCP tool. With just a few lines of code, our MCP server will be ready.

Let's see how to connect it with Cursor.

```
from mcp.server.fastmcp import FastMCP
from agents import run_research

# Create FastMCP instance
mcp = FastMCP("crew_research")

@mcp.tool()
def crew_research(query: str) → str:
    """
    Run CrewAI-based deep-research system for given user query.

    Args:
        query (str): The research query or question.

    Returns:
        str: The research response from the CrewAI pipeline.
    """
    return run_research(query)

# Run the server
if __name__ == "__main__":
    mcp.run(transport="stdio")
```



Model Context Protocol

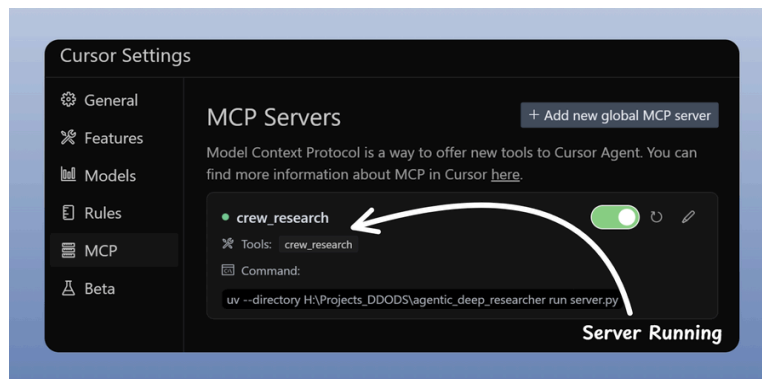
#8) Integrate MCP server with Cursor

Go to: File → Preferences → Cursor Settings → MCP → Add new global MCP server

In the JSON file, add what's shown below



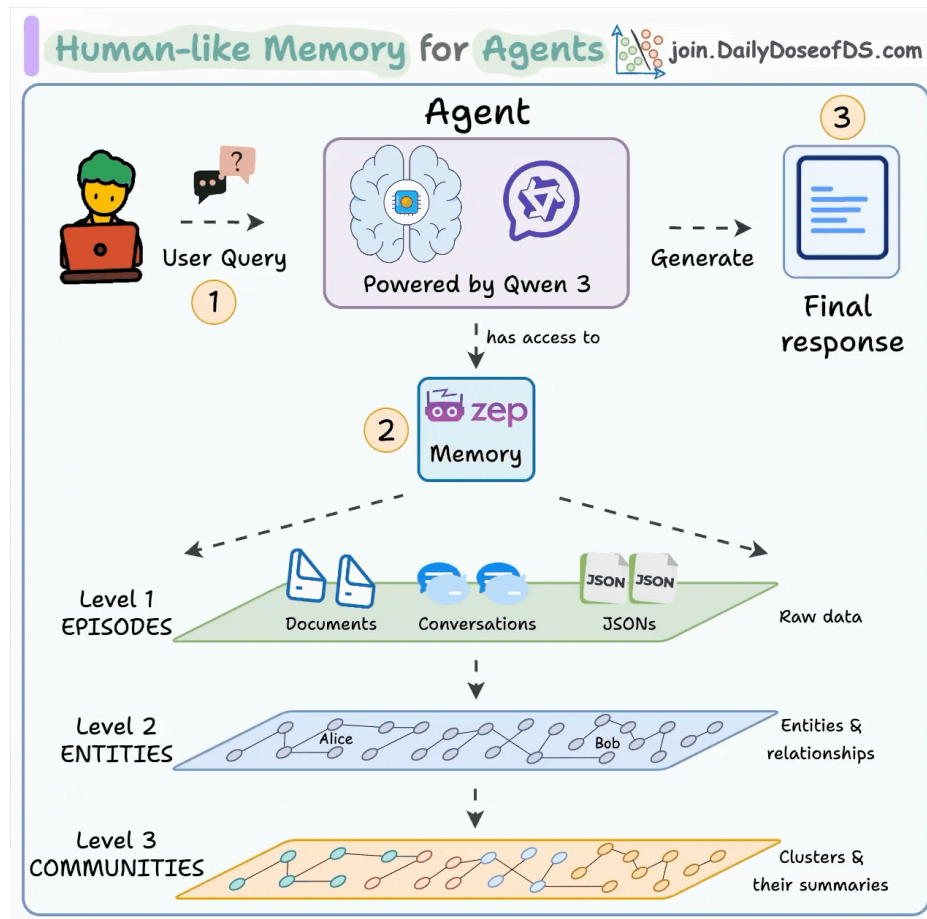
Done! Your deep research MCP server is live and connected to Cursor.



The code is available here:
<https://www.dailydoseofds.com/p/hands-on-mcp-powered-deep-researcher/>

#8) Human-like Memory for Agents

If a memory-less AI Agent is deployed in production, every interaction with the Agent will be a blank slate. Learn how to build an AI Agent with human-like memory to solve this.



Tech stack:

- Zep AI for the memory layer to AI agent
- Microsoft AutoGen for agent orchestration
- Ollama to locally serve Qwen3

Workflow:

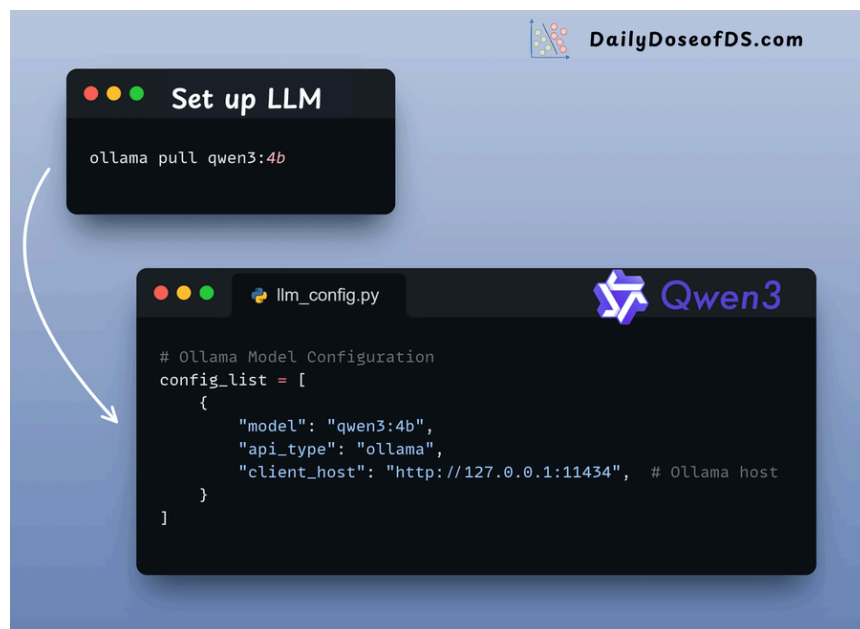
- User submits a query

- Agent saves the conversation and extracts facts into memory
- Agent retrieves facts and summarizes
- Uses facts and history for informed responses

Let's implement this!

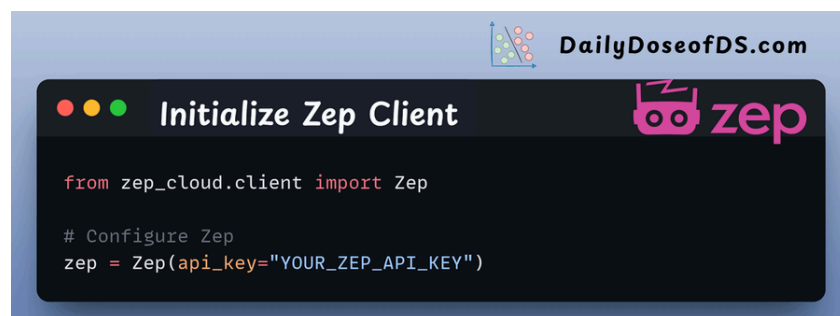
#1) Setup LLM

We'll use a locally served Qwen 3 via Ollama. Check this out:



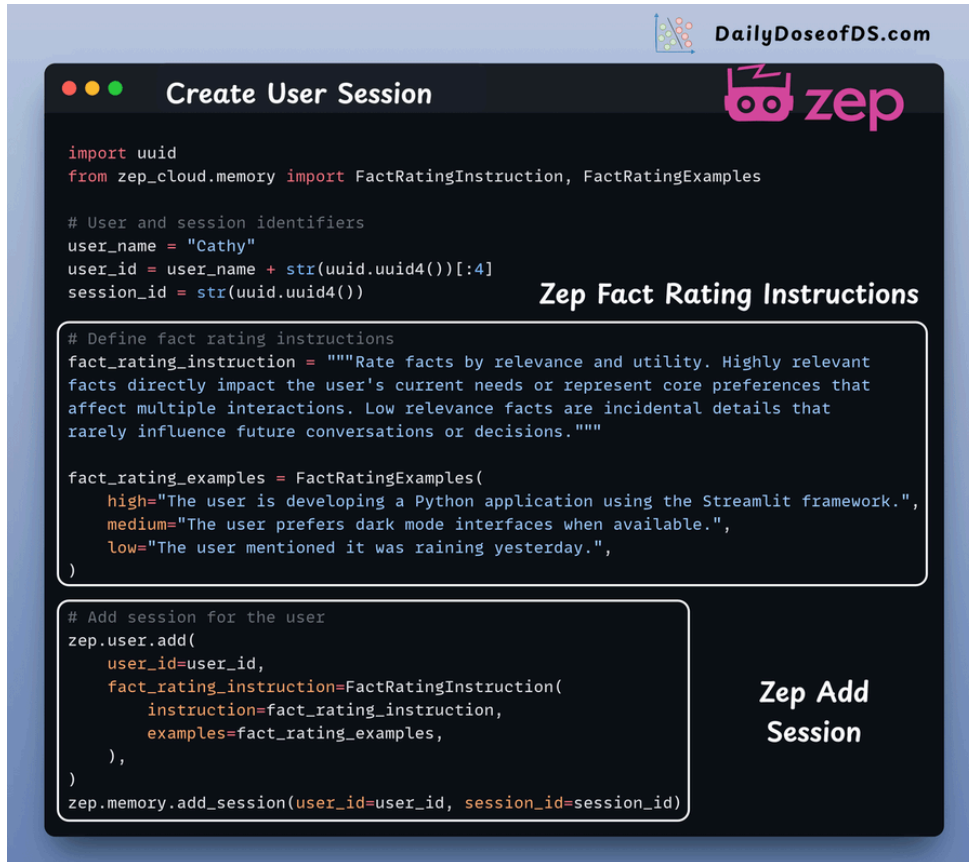
#2) Initialise Zep Client

We're leveraging zep_ai's Foundational Memory Layer to equip our Autogen agent with genuine task-completion capabilities.



#3) Create User Session

Create a Zep client session for the user, which the agent will use to manage memory. A user can have multiple sessions. Here's how it looks:



```
import uuid
from zep_cloud.memory import FactRatingInstruction, FactRatingExamples

# User and session identifiers
user_name = "Cathy"
user_id = user_name + str(uuid.uuid4())[:4]
session_id = str(uuid.uuid4())

# Define fact rating instructions
fact_rating_instruction = """Rate facts by relevance and utility. Highly relevant
facts directly impact the user's current needs or represent core preferences that
affect multiple interactions. Low relevance facts are incidental details that
rarely influence future conversations or decisions."""

fact_rating_examples = FactRatingExamples(
    high="The user is developing a Python application using the Streamlit framework.",
    medium="The user prefers dark mode interfaces when available.",
    low="The user mentioned it was raining yesterday.",
)

# Add session for the user
zep.user.add(
    user_id=user_id,
    fact_rating_instruction=FactRatingInstruction(
        instruction=fact_rating_instruction,
        examples=fact_rating_examples,
    ),
)
zep.memory.add_session(user_id=user_id, session_id=session_id)
```

#4) Define Zep Conversable Agent

Our Zep Memory Agent builds on Autogen's Conversable Agent, drawing live memory context from Zep Cloud with each user query.

It remains efficient by utilizing the session we just established.

Here's how it comes together:



#5) Setting up Agents

We initialize the Conversable Agent and a Stand-in Human Agent to manage chat interactions.

Here's the setup:


DailyDoseofDS.com



Setting up Agents

```

from zep_cloud.client import Zep
from autogen import ConversableAgent
from agent import ZepConversableAgent
from llm_config import config_list

# Create the autogen agent with Zep memory
agent = ZepConversableAgent(
    name="Zep Agent",
    system_message="""
    You are 'Zep Agent', a helpful and professional assistant. Support the
    user by remembering past interactions, offering accurate and clear
    answers, and maintaining a friendly, empathetic tone. Prioritize recent
    context, avoid repeating known info, and never reveal internal workings.
    """,
    llm_config={"config_list": config_list},
    zep_session_id=session_id,
    zep_client=zep,
    min_fact_rating=0.7,
    function_map=None,
    human_input_mode="NEVER",
)

# Create Stand-in Human agent
user = ConversableAgent(
    name="Cathy",
    system_message="""
    You are a helpful mental health bot. You are
    seeking counsel from a mental health bot. Ask
    the bot about your previous conversation.
    """,
    llm_config={"config_list": config_list},
    human_input_mode="NEVER",
)

```

1. Zep
Conversable
Agent

2. Stand-in Human
Agent

#6) Handle Agentic Chat

The Zep Conversable Agent steps in to create a coherent, personalized response.

It seamlessly integrates memory and conversation. Here's how it works:


DailyDoseofDS.com

Handle Agentic Chat

```

result = agent.initiate_chat(
    user,
    message="Hi Cathy, nice to see you again. How are you doing today?",
    max_turns=2,
)

```

Cathy (to CareBot):

Hello CareBot! I'm doing well, thank you for asking. I wanted to reflect on our previous conversation-do you remember what we discussed? It would be helpful to revisit that topic and...

CareBot (to Cathy):

Of course, Cathy. We talked about your difficulty in processing your mother's passing and how you've been feeling down and demotivated lately. It's completely natural to have these feelings...

Cathy (to CareBot):

Cathy: Thank you for reminding me, CareBot. Yes, I've been struggling with those feelings, and it's been tough to navigate. I'd like to explore some coping strategies to help me process...

CareBot (to Cathy):

Absolutely, Cathy. Here are some coping strategies to help you navigate your grief:

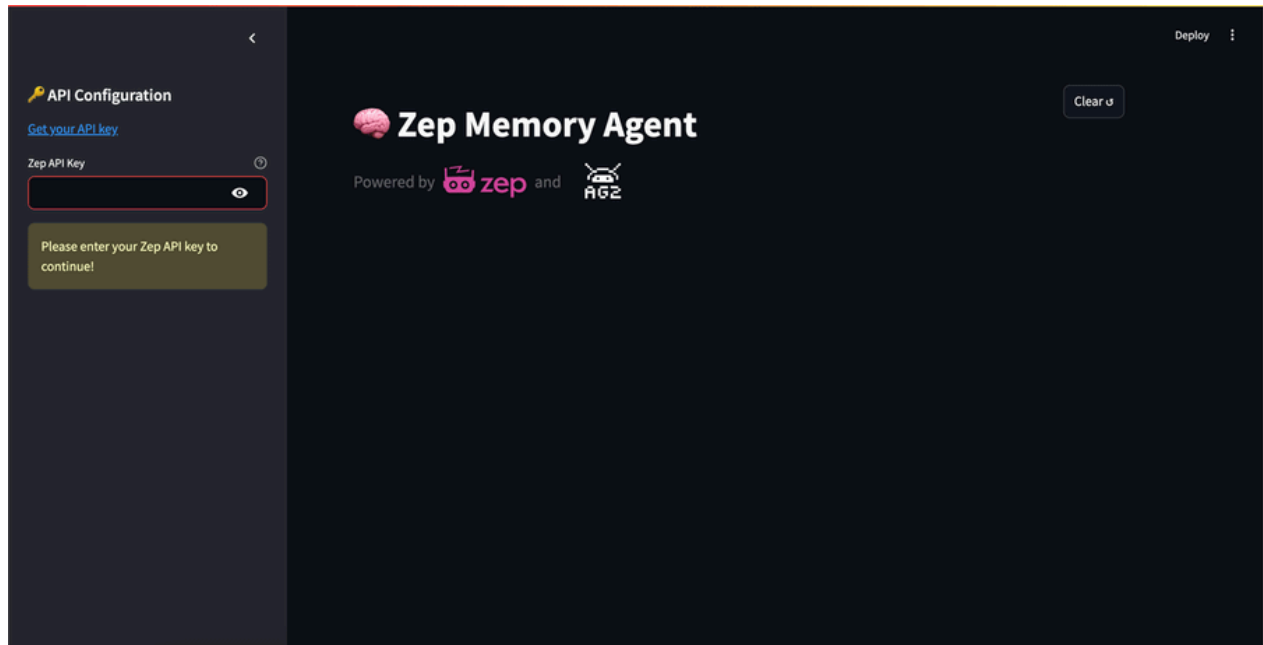
1. **"Journaling"**: Writing your thoughts and feelings can be a great outlet and help you process your emotions.
2. **"Talk to someone"**: Sharing your feelings with a trusted friend or therapist can provide support and understanding.
3. **"Mindfulness and meditation"**: These practices can help ground you and create a sense of calm amid emotional turmoil.
4. **"Create a tribute"**: Honoring your mother through a song, writing letters, or lighting a candle can foster connection and memory.
5. **"Physical activity"**: Engaging in exercise can boost your mood and help alleviate stress.

Remember, it's important to be gentle with yourself as you navigate this process. What resonates with you?

#7) Streamlit UI

We created a streamlined Streamlit UI to ensure smooth and simple interactions with the Agent.

Here's what it looks like:

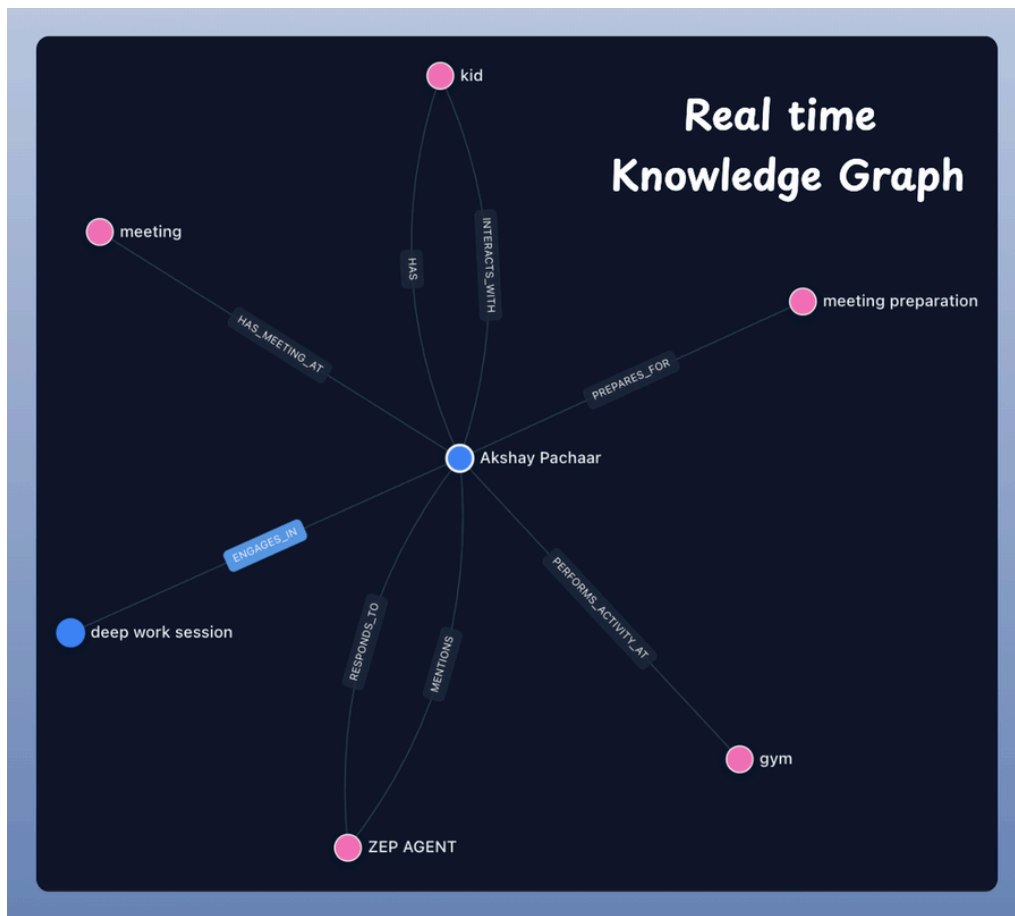


#8) Visualize Knowledge Graph

We can interactively map users' conversations across multiple sessions with Zep Cloud's UI.

This powerful tool allows us to visualize how knowledge evolves through a graph.

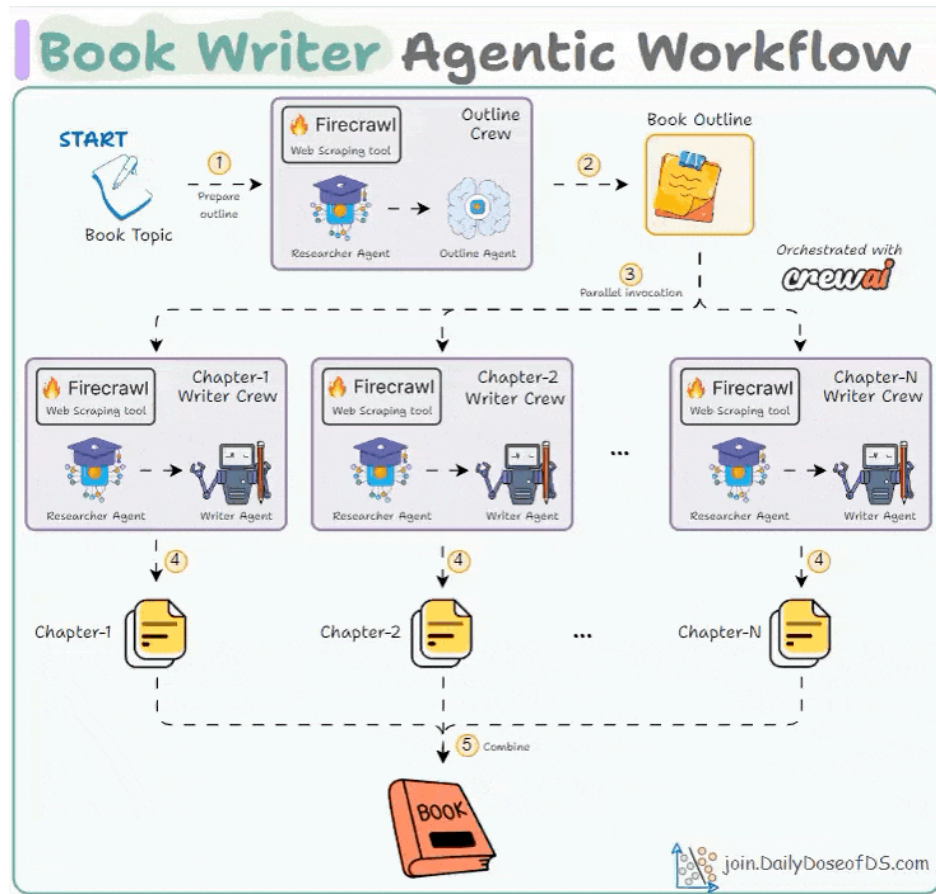
Take a look:



The code is available here:
<https://www.dailydoseofds.com/p/hands-on-building-an-ai-agent-with-human-like-memory/>

#9) Multi-agent Book Writer

Build an Agentic workflow that writes a 20k-word book from a 3-5 word book title.



Tech stack:

- Firecrawl for web scraping.
- CrewAI for orchestration.
- Ollama to serve Qwen 3 locally.
- LightningAI for development and hosting

Workflow:

- Using Firecrawl, Outline Crew scrapes data related to the book title and decides the chapter count and titles.
- Multiple writer Crews work in parallel to write one chapter each.
- Combine all chapters to get the book.

Let's implement this!

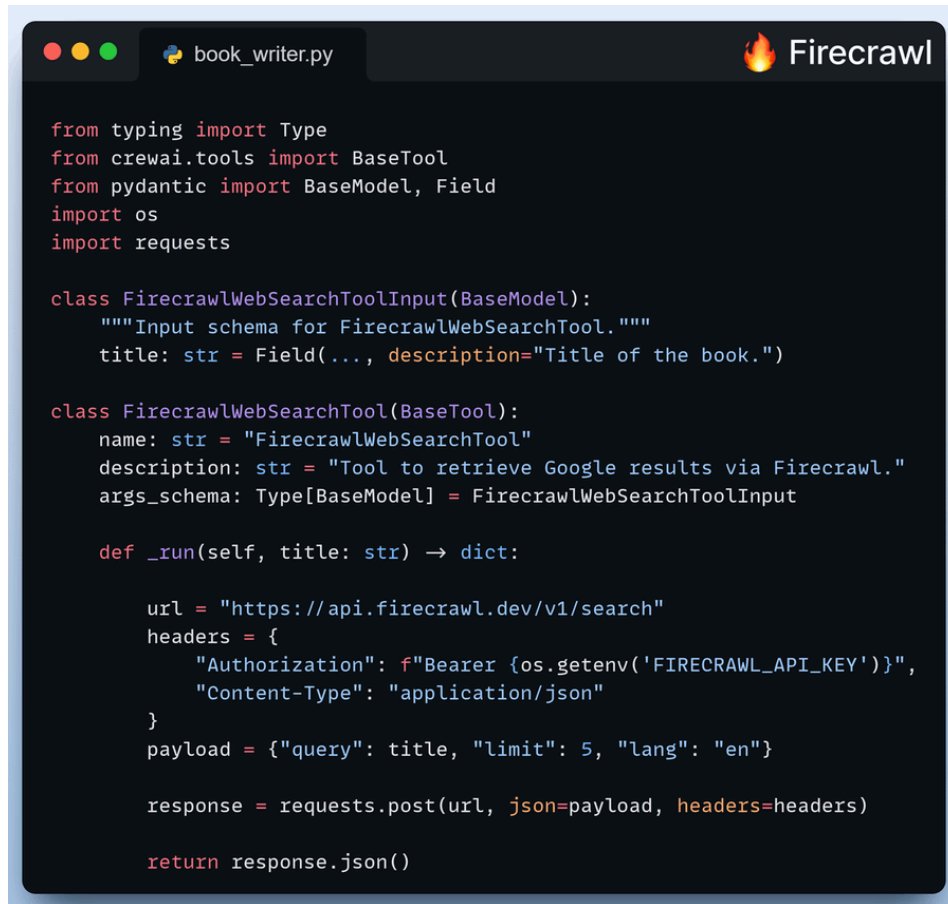
#1) Scraping tool - SERP API

Books demand research. Thus, we'll use Firecrawl's SERP API to scrape data.

Tool usage:

- Outline Crew → to research the book title and prepare an outline.
- Writer Crew → to research the chapter title and write it.

See this code:

A screenshot of a code editor window titled 'book_writer.py' with a 'Firecrawl' logo in the top right corner. The code defines a Pydantic model and a CrewAI tool for searching the web using Firecrawl. The code is as follows:

```
from typing import Type
from crewai.tools import BaseTool
from pydantic import BaseModel, Field
import os
import requests

class FirecrawlWebSearchToolInput(BaseModel):
    """Input schema for FirecrawlWebSearchTool."""
    title: str = Field(..., description="Title of the book.")

class FirecrawlWebSearchTool(BaseTool):
    name: str = "FirecrawlWebSearchTool"
    description: str = "Tool to retrieve Google results via Firecrawl."
    args_schema: Type[BaseModel] = FirecrawlWebSearchToolInput

    def _run(self, title: str) -> dict:

        url = "https://api.firecrawl.dev/v1/search"
        headers = {
            "Authorization": f"Bearer {os.getenv('FIRECRAWL_API_KEY')}",
            "Content-Type": "application/json"
        }
        payload = {"query": title, "limit": 5, "lang": "en"}

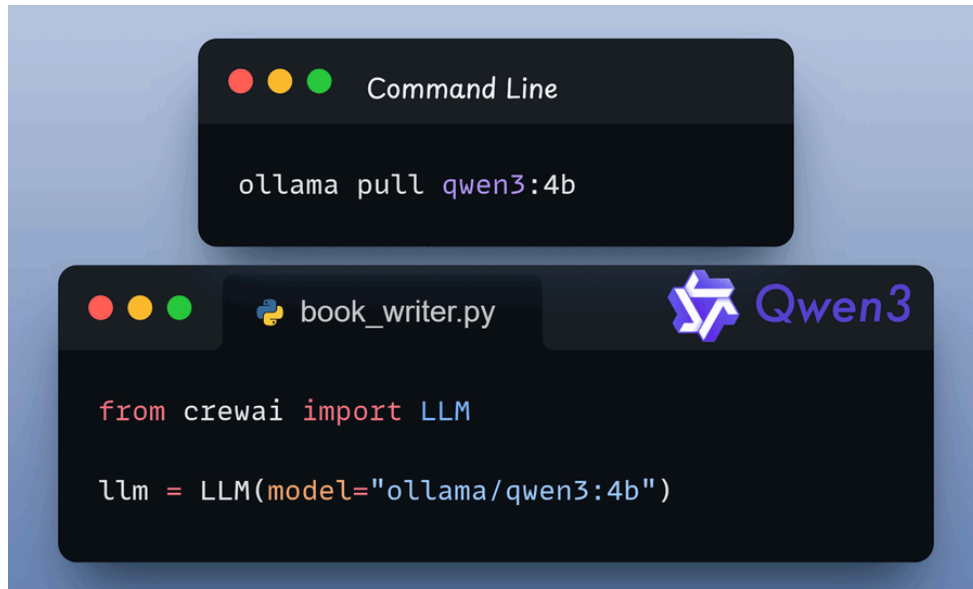
        response = requests.post(url, json=payload, headers=headers)

        return response.json()
```

#2) Setup Qwen 3 locally

We'll serve Qwen 3 locally through Ollama. To do this:

- First, we download it locally.
- Next, we define it with the CrewAI's LLM class.



#3) Outline Crew

This Crew has two Agents:

- Research Agent → Uses the Firecrawl scraping tool to scrape data related to the book's title and prepare insights.
- Outline Agent → Uses the insights to output total chapters and titles as Pydantic output.



#4) Writer Crew

This Crew has two Agents:

- Research Agent → Uses the Firecrawl Scraping tool to scrape data related to a chapter's title and prepare insights.
- Write Agent → Uses the insights to write a chapter.

Check this code:



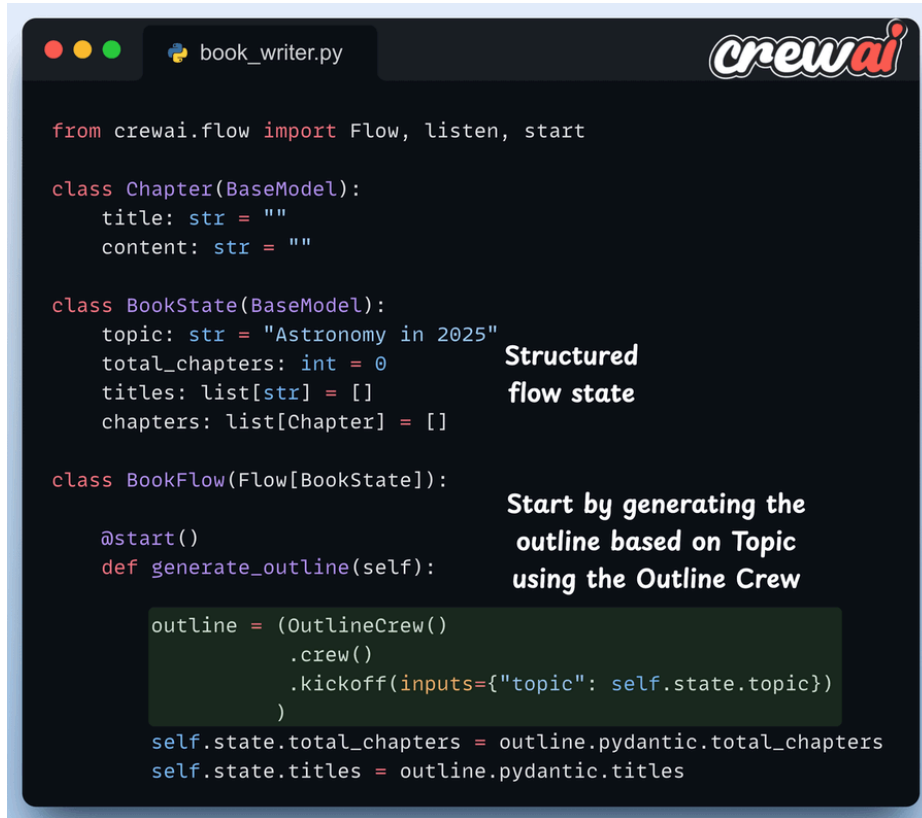
#5) Create a Flow

We use CrewAI Flows to orchestrate the workflow.

First, the outline method invokes the Outline Crew, which:

- research the topic using the scraping tool.
- returns the total number of chapters and the corresponding titles.

This is implemented below:



```
from crewai.flow import Flow, listen, start

class Chapter(BaseModel):
    title: str = ""
    content: str = ""

class BookState(BaseModel):
    topic: str = "Astronomy in 2025"
    total_chapters: int = 0
    titles: list[str] = []
    chapters: list[Chapter] = []

class BookFlow(Flow[BookState]):

    @start()
    def generate_outline(self):
        outline = (OutlineCrew()
                   .crew()
                   .kickoff(inputs={"topic": self.state.topic})
                  )
        self.state.total_chapters = outline.pydantic.total_chapters
        self.state.titles = outline.pydantic.titles
```

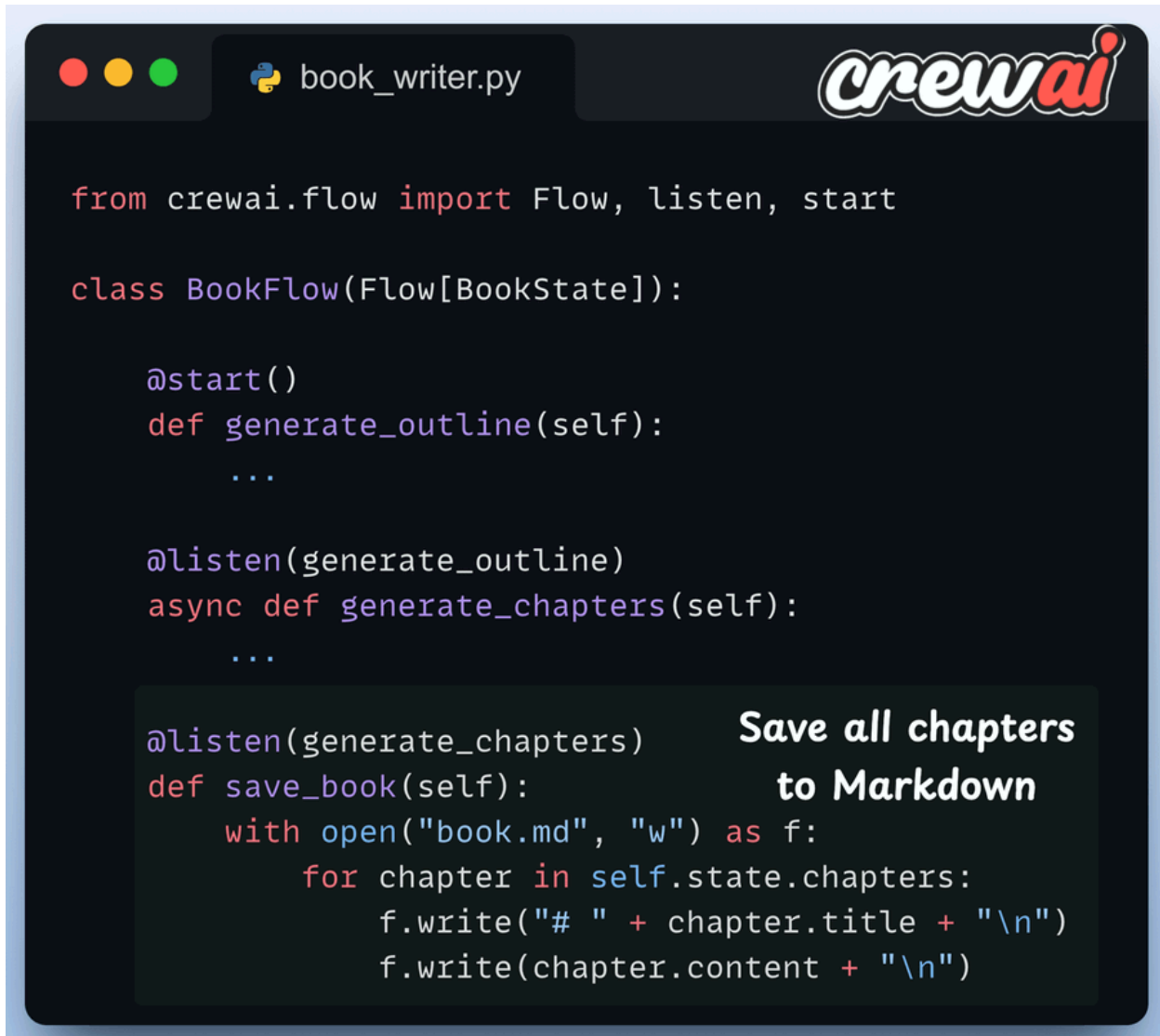
Structured flow state

Start by generating the outline based on Topic using the Outline Crew

#6) Save the book

Once all Writer Crews have finished execution, we save the book as a Markdown file.

Check this code:



```
from crewai.flow import Flow, listen, start

class BookFlow(Flow[BookState]):

    @start()
    def generate_outline(self):
        ...

    @listen(generate_outline)
    async def generate_chapters(self):
        ...

    @listen(generate_chapters)
    def save_book(self):
        with open("book.md", "w") as f:
            for chapter in self.state.chapters:
                f.write("# " + chapter.title + "\n")
                f.write(chapter.content + "\n")
```

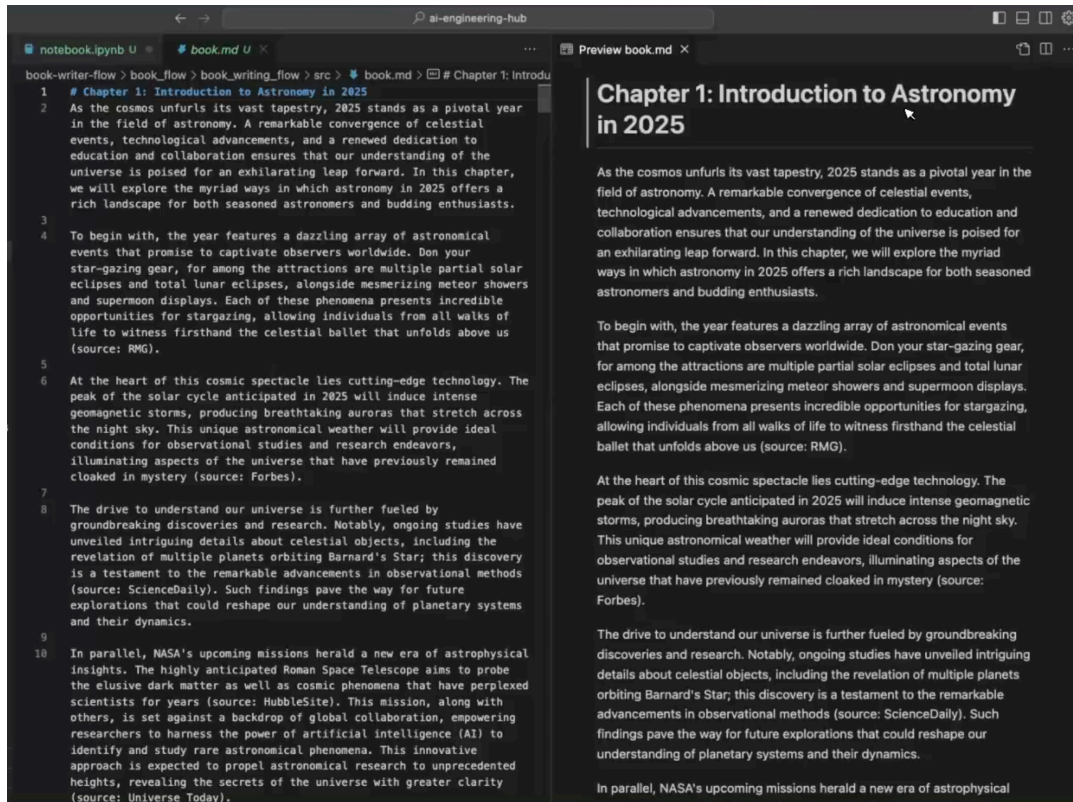
Save all chapters to Markdown

#7) Kickoff the Flow

Finally, we run the Flow.

- First, the Outline Crew is invoked. It utilizes the Firecrawl scraping tool to prepare the outline.
- Next, many Writer Crews are invoked in parallel to write one chapter each.

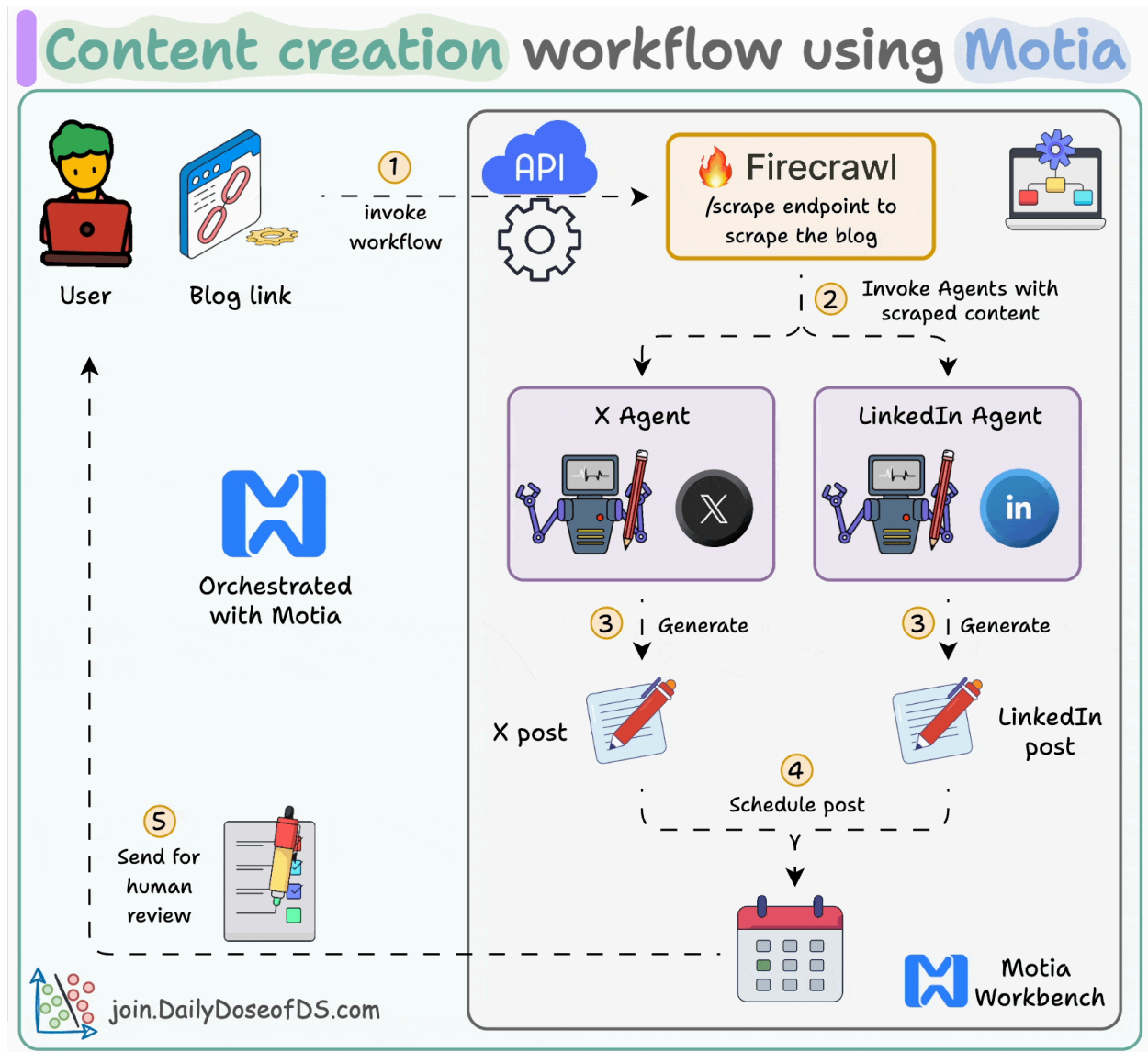
This workflow runs for ~2 minutes, and we get a neatly written book about the specified topic—"Astronomy in 2025." Here's the book our Agentic workflow wrote:



The code is available here:
<https://blog.dailydoseofds.com/p/building-a-multi-agent-book-writer/>

#10) Multi-agent Content Creation System

Build an Agentic workflow that turns any URL into social media posts and auto-schedules them via Typefully.



Tech stack:

- Motia as the unified backend framework
- Firecrawl to scrape web content
- Ollama to locally serve Deepseek-R1 LLM

Workflow:

- User submits URL to scrape
- Firecrawl scrapes content and converts it to markdown
- Twitter and LinkedIn agents run in parallel to generate content
- Generated content gets scheduled via Typefully

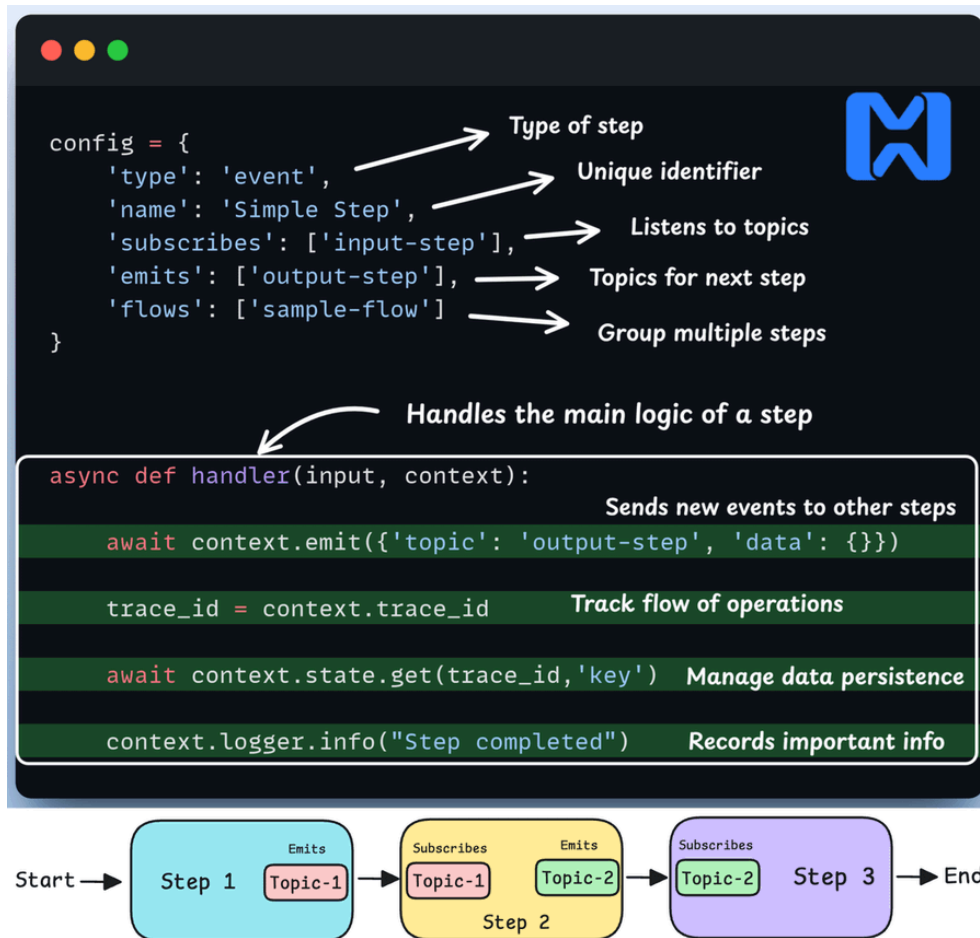
Let's implement this:

Steps are the fundamental building blocks of Motia.

They consist of two main components:

- The Config object: It instructs Motia on how to interact with a step.
- The handler function: It defines the main logic of a step.

Check this out:



With that understanding in mind, let's start building our content creation workflow.

#1) Entry point (API)

We start our content generation workflow by defining an API step that takes in a URL from the user via a POST request.

Check this out:



```
api.step.py

config = {
    'type': 'api',
    'name': 'ContentGenerationAPI',
    'description': 'Triggers content generation from URL',
    'path': '/generate-content',
    'method': 'POST',
    'emits': ['scrape-article'],
    'flows': ['content-generation']
}

async def handler(req, context):
    await context.emit({
        'topic': 'scrape-article',
        'data': {
            'requestId': context.traceId,
            'url': req['body']['url']
        }
    })

    return {
        'status': 200,
        'body': {
            'message': 'Content generation started',
            'requestId': context.traceId,
            'url': req['body']['url'],
            'status': 'processing'
        }
    }
```

Workflow initiated successfully

#2) Web scraping

This step scrapes the article content using Firecrawl and emits the next step in the workflow.

Steps can be connected together in a sequence, where the output of one step becomes the input for another.

Check this out:



```
from firecrawl import FirecrawlApp

config = {
    'type': 'event',
    'name': 'ScrapeArticle',
    'description': 'Scrapes content using Firecrawl',
    'subscribes': ['scrape-article'],
    'emits': ['generate-content'],
    'input': ScrapeInput,
    'flows': ['content-generation']
}

async def handler(input, context):
    context.logger.info(f"Scraping article: {input['url']}")

    app = FirecrawlApp(api_key=FIRECRAWL_API_KEY)
    scrapeResult = await app.scrape_url(input['url'])

    await context.emit({
        'topic': 'generate-content',
        'data': {
            'requestId': input['requestId'],
            'url': str(input['url']),
            'title': scrapeResult.metadata.get('title'),
            'content': scrapeResult.markdown,
        }
    })
```

Sent as input to the next step

#3) Content generation

The scraped content gets fed to the X and LinkedIn agents that run in parallel and generate curated posts.

We define all our prompting and AI logic in the handler that runs automatically when a step is triggered.

Check this out:

```
generate.step.py

import asyncio
from ollama import AsyncClient

config = {
    'type': 'event',
    'name': 'GenerateContent',
    'description': 'Generates social media content',
    'subscribes': ['generate-content'],
    'emits': ['schedule-content'],
    'input': GenerateInput,
    'flows': ['content-generation']
}

async def handler(input, context):
    twitter_resp, linkedin_resp = await asyncio.gather(
        AsyncClient().chat(model="deepseek-r1",
            messages=[{
                'role': 'user',
                'content': twitter_prompt
            }]),
        AsyncClient().chat(model="deepseek-r1",
            messages=[{
                'role': 'user',
                'content': linkedin_prompt
            }])

    await context.emit({
        'topic': 'schedule-content',
        'data': { 'content': {twitter_content, linkedin_content} }
    })
```

Both agents invoked in parallel

X Agent



LinkedIn Agent



#4) Scheduling

After the content is generated we draft it in Typefully where we can easily review our social media posts.

Motia also allows us to mix and match different languages within the same workflow providing great flexibility.

Check this typescript code:



```
import { EventConfig, Handlers } from 'motia'
import axios from 'axios'

export const config: EventConfig = {
  type: 'event',
  name: 'Schedule',
  description: 'Schedules social media posts using Typefully',
  subscribes: ['schedule-content'],
  emits: [],
  flows: ['content-generation']
}

export const handler: Handlers['Schedule'] = async (input, {logger}) => {
  const typefullyHeaders = {
    'X-API-KEY': `Bearer ${TYPEFULLY_API_KEY}`,
    'Content-Type': 'application/json'
  }

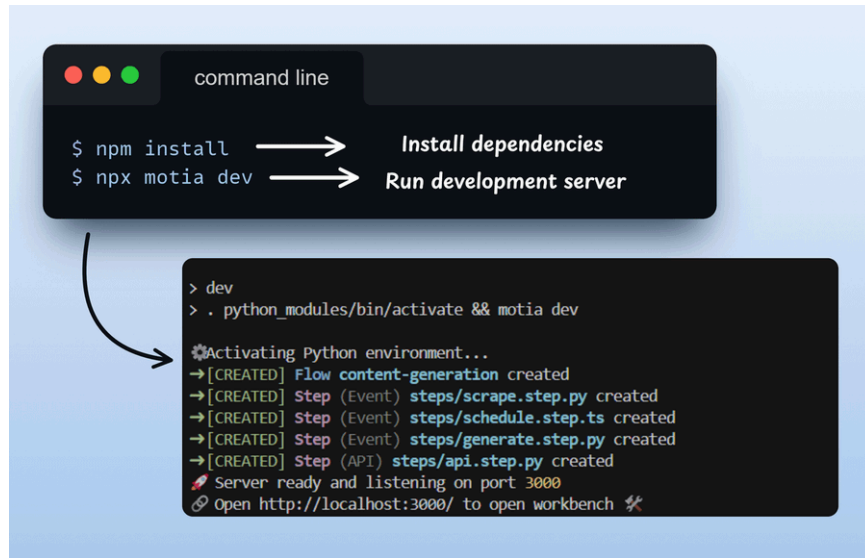
  await axios.post('https://api.typefully.com/v1/drafts/', {
    content: twitterTweets.join('\n\n\n\n'),
    schedule_date: twitterScheduleTime
  }, { headers: typefullyHeaders })

  await axios.post('https://api.typefully.com/v1/drafts/', {
    content: input.content.linkedin.post,
    schedule_date: linkedinScheduleTime,
  }, { headers: typefullyHeaders })

  logger.info(`Content scheduling completed successfully!\n`)
}
```

After defining our steps, we install required dependencies using `npm install` and run the Motia workbench using `npm run dev` commands.

Check this out:



```
command line
$ npm install → Install dependencies
$ npx motia dev → Run development server

> dev
> . python_modules/bin/activate && motia dev

Activating Python environment...
→[CREATED] Flow content-generation created
→[CREATED] Step (Event) steps/scrape.step.py created
→[CREATED] Step (Event) steps/schedule.step.ts created
→[CREATED] Step (Event) steps/generate.step.py created
→[CREATED] Step (API) steps/api.step.py created
🔥 Server ready and listening on port 3000
🔗 Open http://localhost:3000/ to open workbench 🔥
```

Motia workbench provides an interactive UI to help build, monitor and debug our flows.

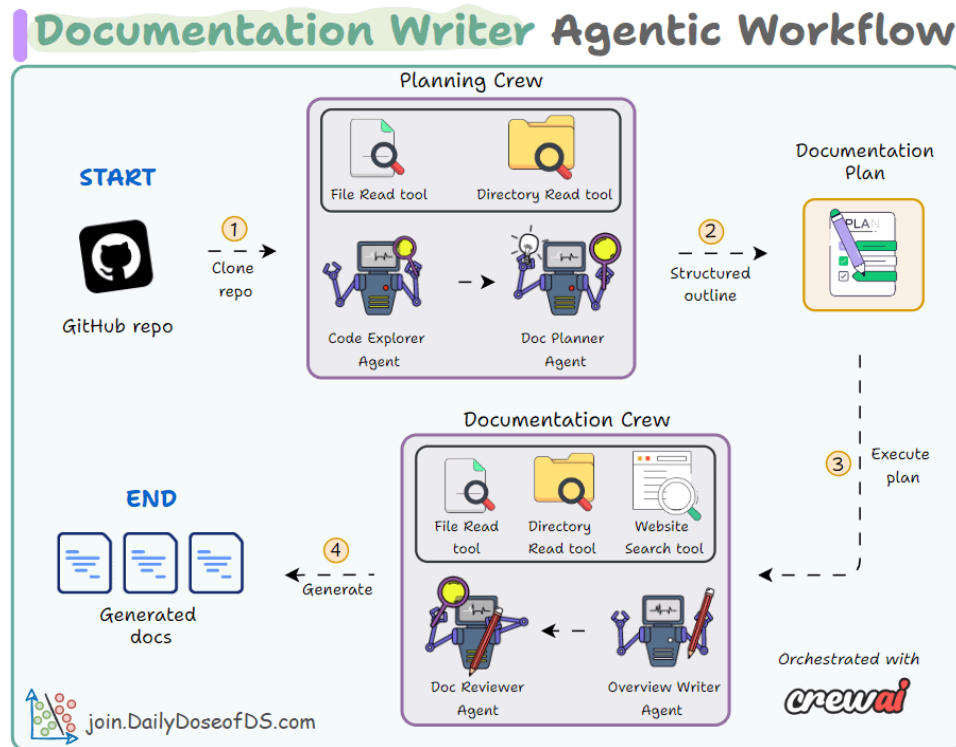
With one-click you can also deploy it to the cloud!



The code is available here:
<https://blog.dailydoseofds.com/p/build-a-multi-agent-content-creation/>

#11) Documentation Writer Flow

Build an Agentic workflow that generates full project documentation from just a GitHub repo URL.



Tech stack:

- CrewAI for multi-agent orchestration
- Ollama to locally serve DeepSeek-R1 LLM

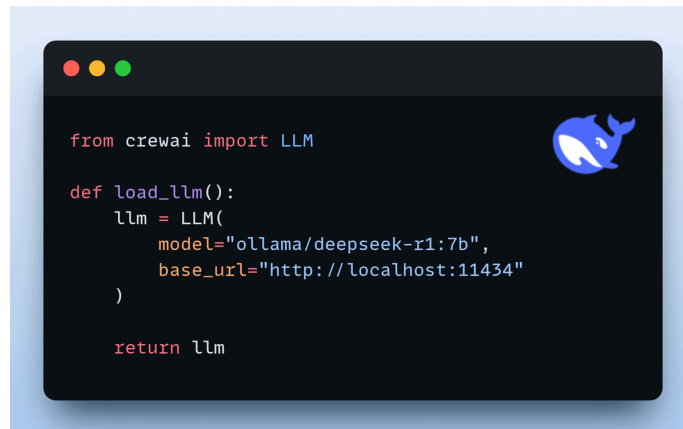
Workflow:

- User specifies GitHub repo
- Planning crew creates documentation plan
- Documentation crew writes documentation according to plan
- Generated docs get saved to local directory

Let's implement this!

#1) Setup LLM

We will use Deepseek-R1 as the LLM, served locally using Ollama.



```
from crewai import LLM

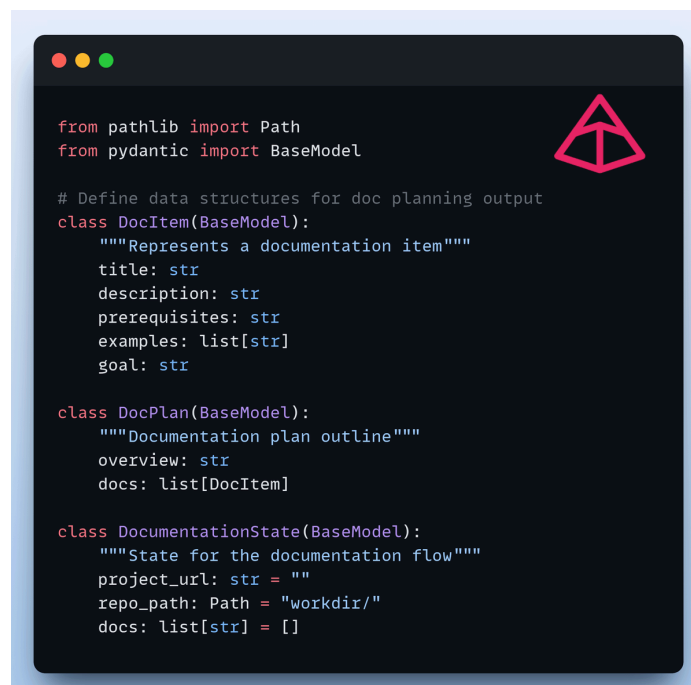
def load_llm():
    llm = LLM(
        model="ollama/deepseek-r1:7b",
        base_url="http://localhost:11434"
    )

    return llm
```

#2) Define Pydantic schema

We define the following pydantic schemas for robust structured outputs.

This ensures data validation and integrity before generating the documentation files. Check this code:



```
from pathlib import Path
from pydantic import BaseModel

# Define data structures for doc planning output
class DocItem(BaseModel):
    """Represents a documentation item"""
    title: str
    description: str
    prerequisites: str
    examples: list[str]
    goal: str

class DocPlan(BaseModel):
    """Documentation plan outline"""
    overview: str
    docs: list[DocItem]

class DocumentationState(BaseModel):
    """State for the documentation flow"""
    project_url: str = ""
    repo_path: Path = "workdir/"
    docs: list[str] = []
```


#3) Planning Crew

This crew oversees strategy for the outline via:

- Code explorer agent -> Analyzes codebase for key components, patterns and relationships.
- Doc planner agent -> Creates outline based on codebase analysis as pydantic output.

Check this out:

```
from crewai import Agent, Task, Crew
from crewai_tools import (
    DirectoryReadTool, FileReadTool
)

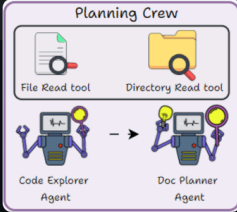
code_explorer = Agent(role="Code Explorer",
                      goal="Analyze codebase and identify key components",
                      backstory="Expert in breaking down large codebases",
                      tools=[DirectoryReadTool(), FileReadTool()])

analyze_code = Task(description="Analyze the codebase at {repo_path}",
                    expected_output="Technical summary with code examples, "
                                   "guidelines and design patterns",
                    agent=code_explorer)

doc_planner = Agent(role="Documentation Planner",
                    goal="Create documentation strategy based on analysis",
                    backstory="Expert in organizing technical documentation",
                    tools=[DirectoryReadTool(), FileReadTool()])

create_plan = Task(description="Create a documentation plan for {repo_path}"
                  expected_output="Documentation plan with list of topics",
                  agent=doc_planner,
                  output_pydantic=DocPlan)

planning_crew = Crew(agents=[code_explorer, doc_planner],
                    tasks=[analyze_code, create_plan])
```



The diagram titled "Planning Crew" illustrates the workflow between two agents. At the top, there are two tool icons: "File Read tool" and "Directory Read tool". Below these, the "Code Explorer Agent" is shown on the left and the "Doc Planner Agent" is shown on the right. An arrow points from the Code Explorer Agent to the Doc Planner Agent, indicating the flow of information or task delegation. The agents are depicted as stylized robot figures.

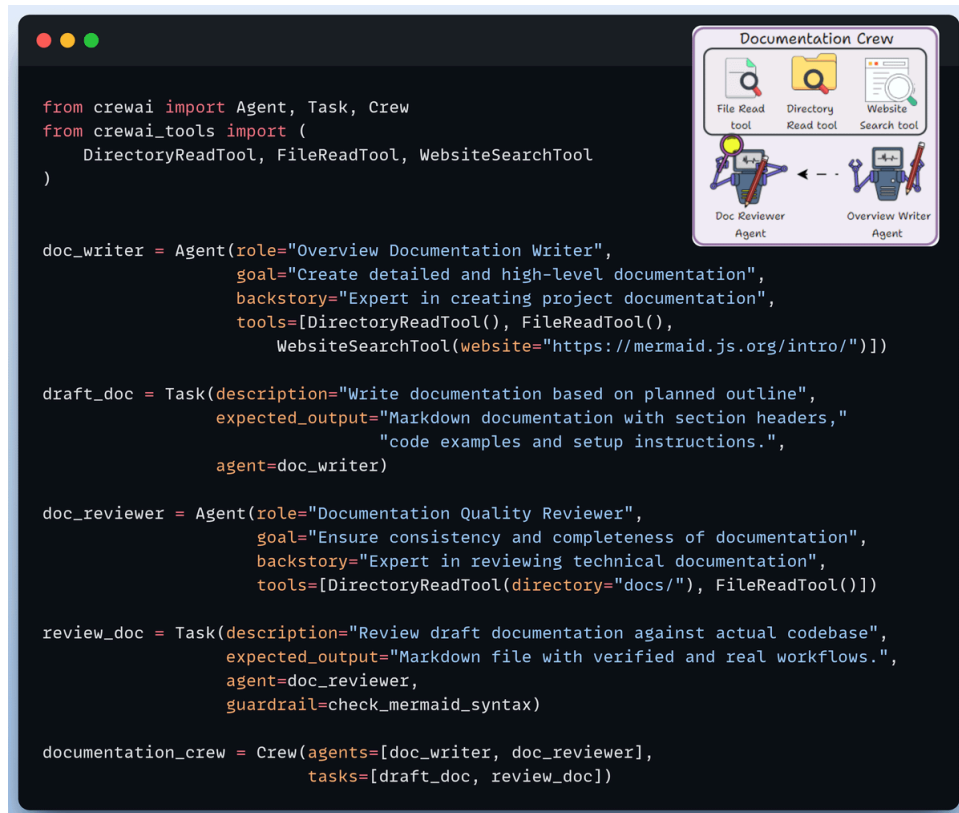
#4) Documentation Crew

This crew writes and reviews the documentation via:

- Doc writer agent -> Generates a high-level draft based on the planned outline.

- Doc reviewer agent -> Reviews the draft for consistency, accuracy and completeness.

Check this out:



```
from crewai import Agent, Task, Crew
from crewai_tools import (
    DirectoryReadTool, FileReadTool, WebsiteSearchTool
)

doc_writer = Agent(role="Overview Documentation Writer",
    goal="Create detailed and high-level documentation",
    backstory="Expert in creating project documentation",
    tools=[DirectoryReadTool(), FileReadTool(),
    WebsiteSearchTool(website="https://mermaid.js.org/intro/")])

draft_doc = Task(description="Write documentation based on planned outline",
    expected_output="Markdown documentation with section headers,"
    "code examples and setup instructions.",
    agent=doc_writer)

doc_reviewer = Agent(role="Documentation Quality Reviewer",
    goal="Ensure consistency and completeness of documentation",
    backstory="Expert in reviewing technical documentation",
    tools=[DirectoryReadTool(directory="docs/"), FileReadTool()])

review_doc = Task(description="Review draft documentation against actual codebase",
    expected_output="Markdown file with verified and real workflows.",
    agent=doc_reviewer,
    guardrail=check_mermaid_syntax)

documentation_crew = Crew(agents=[doc_writer, doc_reviewer],
    tasks=[draft_doc, review_doc])
```

The image also includes a small diagram titled "Documentation Crew" showing the workflow. It features three tool icons at the top: "File Read tool", "Directory Read tool", and "Website Search tool". Below these are two agent icons: "Doc Reviewer Agent" and "Overview Writer Agent". Arrows indicate a flow from the tools to the agents, and from the Overview Writer Agent to the Doc Reviewer Agent.

#5) Create Documentation Flow

After setting up our crews, we create the main workflow that:

- Clones the GitHub repo
- Plans and saves the outline
- Generates documentation based on the outline
- Saves final docs to local directory

Check this code:

```
import subprocess
from crewai.flow.flow import Flow, listen, start

class CreateDocumentationFlow(Flow[DocumentationState]):
    @start()
    def clone_repo(self):
        """Clone the GitHub repository"""
        subprocess.run(["git", "clone", self.state.project_url, self.state.repo_path])
        return self.state

    @listen(clone_repo)
    def plan_docs(self):
        """Creates documentation outline"""
        result = planning_crew.kickoff(inputs={'repo_path': self.state.repo_path})
        return result

    @listen(plan_docs)
    def save_plan(self, plan):
        """Saves the planned outline for next crew"""
        with open("docs/plan.json", "w") as f:
            f.write(plan.raw)

    @listen(plan_docs)
    def create_docs(self, plan):
        """Generate and save final docs to local directory"""
        for doc in plan.pydantic.docs:
            result = documentation_crew.kickoff(inputs={...})

            with open(f"docs/{title}", "w") as f:
                f.write(result.raw)
            return self.state.docs
```

#6) Kickoff the flow

Finally, when we have everything ready, we kick off our documentation flow with the GitHub repo URL.

Check this out:

```
repo_url = "https://github.com/username/repo"

flow = CreateDocumentationFlow()

flow.kickoff(inputs={"project_url": repo_url})
```

Flow Execution

```
Starting Flow Execution
Name: CreateDocumentationFlow
ID: 0f54b988-af69-4dfb-9433-158f3feeb42f

Flow: CreateDocumentationFlow
ID: 0f54b988-af69-4dfb-9433-158f3feeb42f
└─ Starting Flow...

Flow started with ID: 0f54b988-af69-4dfb-9433-158f3feeb42f
Flow: CreateDocumentationFlow
ID: 0f54b988-af69-4dfb-9433-158f3feeb42f
└─ Starting Flow...
└─ Running: clone_repo

# Cloning repository: https://github.com/nikhil-1e9/machine-learning-daily

Cloning into 'workdir/machine-learning-daily'...
remote: Enumerating objects: 621, done.
remote: Counting objects: 100% (621/621), done.
remote: Compressing objects: 100% (516/516), done.
remote: Total 621 (delta 135), reused 570 (delta 87), pack-reused 0 (from 0)
Receiving objects: 100% (621/621), 1.74 MiB | 2.32 MiB/s, done.
Resolving deltas: 100% (135/135), done.
Flow: CreateDocumentationFlow
ID: 0f54b988-af69-4dfb-9433-158f3feeb42f
└─ Flow Method Step
└─ Completed: clone_repo

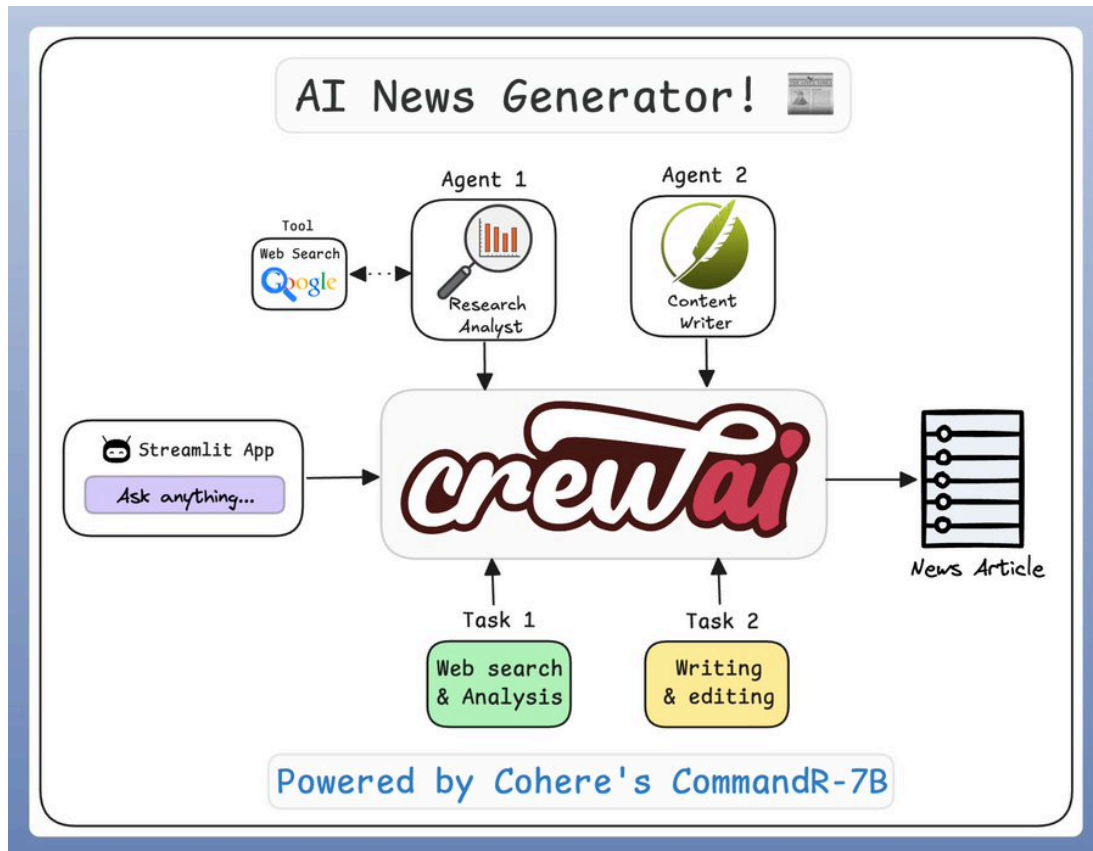
Flow: CreateDocumentationFlow
ID: 0f54b988-af69-4dfb-9433-158f3feeb42f
└─ Flow Method Step
└─ Completed: clone_repo
└─ Running: plan_docs
```



The code is available here:
<https://github.com/patchy631/ai-engineering-hub/tree/main/documentation-writer-flow>

#12) News Generator

The app takes a user query, searches the web for it, and turns it into a well-crafted news article, with citations.



Tech stack:

- Cohere ultra-fast Command R 7B as LLM
- CrewAI for multi-agent orchestration

Workflow:

We'll have two agents in this multi-agent app:

Research analyst agent:

- Accepts a user query.
- Uses the Serper web search tool to fetch results from the internet.
- Consolidates the results.

Content writer agent:

- Uses the curated results to prepare a polished, publication-ready article.

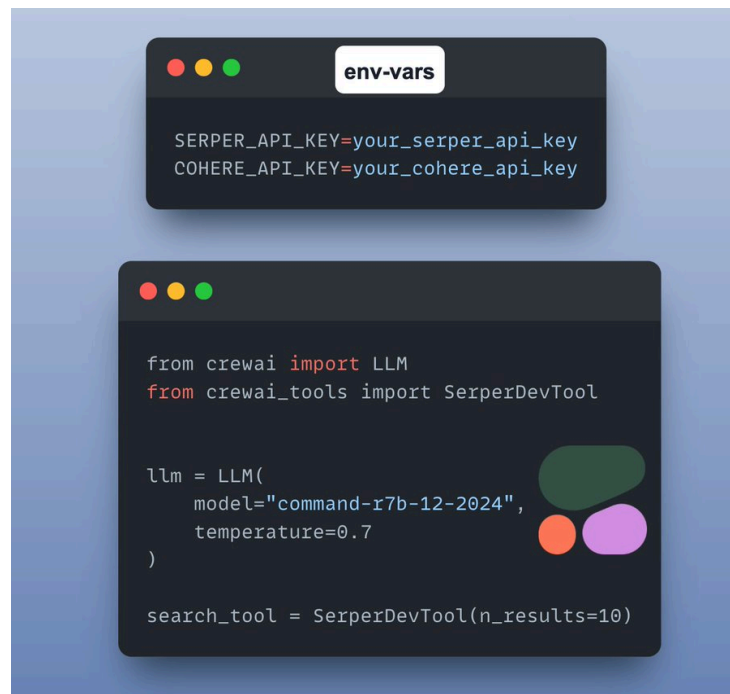
Let's implement this:

#1) Setup

Create a .env file for their corresponding API keys:

- Cohere API key
- Serper API key

Next, setup the LLM and web search tool as follows:



```
env-vars
SERPER_API_KEY=your_serper_api_key
COHERE_API_KEY=your_cohere_api_key

from crewai import LLM
from crewai_tools import SerperDevTool

llm = LLM(
    model="command-r7b-12-2024",
    temperature=0.7
)

search_tool = SerperDevTool(n_results=10)
```

#2) Senior Research Analyst Agent

The web-search agent takes a user query and then uses the Serper web search tool to fetch results from the internet and consolidate them. Check this out:

```
from crewai import Agent

senior_research_analyst = Agent(
    role="Senior Research Analyst",
    goal="Research, analyze, and synthesize comprehensive information on {topic} from \n"
        "reliable web sources",
    backstory="You're an expert research analyst with advanced web research skills. "
        "You excel at finding, analyzing, and synthesizing information from "
        "across the internet using search tools. You're skilled at "
        "distinguishing reliable sources from unreliable ones, "
        "fact-checking, cross-referencing information, and "
        "identifying key patterns and insights. You provide "
        "well-organized research briefs with proper citations "
        "and source verification. Your analysis includes both "
        "raw data and interpreted insights, making complex "
        "information accessible and actionable.",
    allow_delegation=False,
    verbose=True,
    tools=[search_tool],
    llm=llm
)
```

A circular icon with a magnifying glass over a bar chart, with the text "Research Analyst" below it.

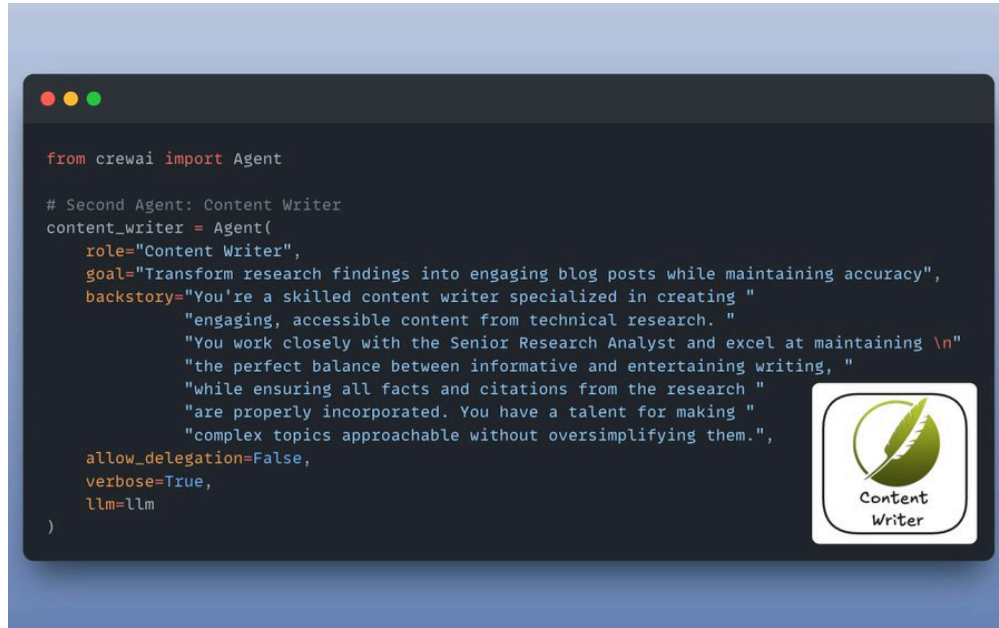
This is the research task that we assign to our senior research analyst agent, with description and expected output.

```
from crewai import Task

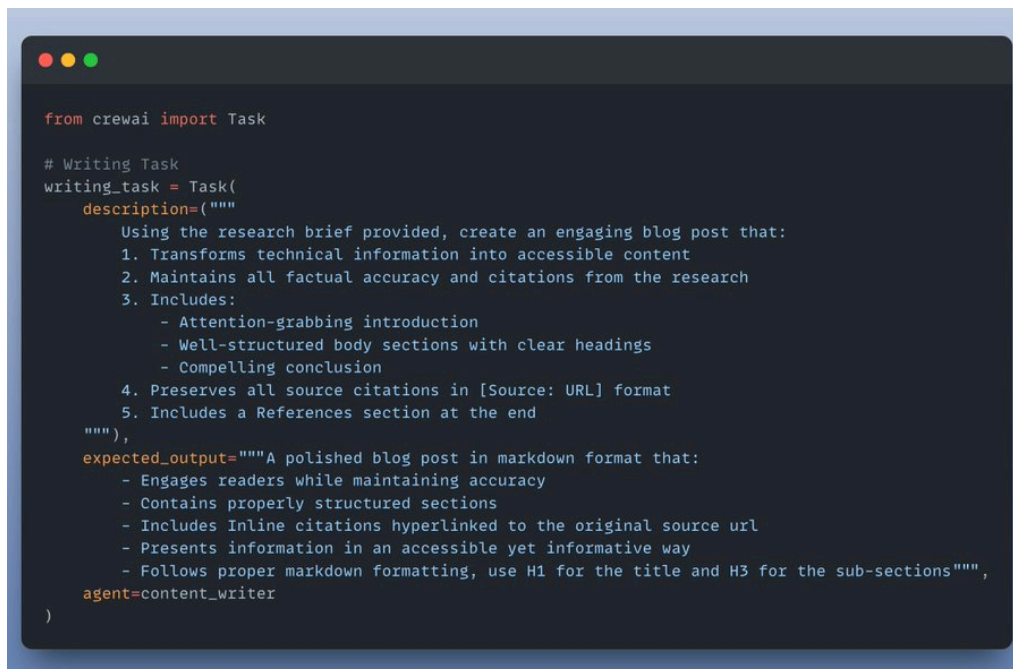
research_task = Task(
    description="""
        1. Conduct comprehensive research on {topic} including:
            - Recent developments and news
            - Key industry trends and innovations
            - Expert opinions and analyses
            - Statistical data and market insights
        2. Evaluate source credibility and fact-check all information
        3. Organize findings into a structured research brief
        4. Include all relevant citations and sources
    """,
    expected_output="""A detailed research report containing:
        - Executive summary of key findings
        - Comprehensive analysis of current trends and developments
        - List of verified facts and statistics
        - All citations and links to original sources
        - Clear categorization of main themes and patterns
        Please format with clear sections and bullet points for easy reference.""",
    agent=senior_research_analyst
)
```

#3) Content writer agent

The role of content writer is to use the curated results and turn them into a polished, publication-ready news article .



This is how we describe the writing task with all the details and expected output:



#4) Setup Crew

And we're done!

Just build a crew and kick it off!

The screenshot displays a Jupyter Notebook interface. The top section contains Python code for creating a CrewAI crew and executing it. The code defines two agents: a senior research analyst and a content writer, each with specific tasks. The crew is then kicked off with the topic "AI Trends in 2024". Below the code, a diagram illustrates the workflow: a topic is input into the CrewAI system, which then assigns tasks to two agents. Agent 1 (Research Analyst) uses a web search tool to gather information, while Agent 2 (Content Writer) uses the gathered information to write a news article. The final output is a markdown document titled "Unlocking the Future: AI Trends to Watch in 2024".

```
from crewai import Crew
from IPython.display import Markdown

# Create Crew
crew = Crew(
    agents=[senior_research_analyst, content_writer],
    tasks=[research_task, writing_task],
    verbose=True
)

result = crew.kickoff(inputs={"topic": "AI Trends in 2024"})

Markdown(result.raw)
```

Unlocking the Future: AI Trends to Watch in 2024

Artificial Intelligence (AI) is no longer the stuff of science fiction. Several key trends are set to redefine the landscape of AI, making the pressing need for strong governance frameworks, let's explore them.

The Rise of Generative AI: A Creative Revolution

Generative AI is leading the AI revolution with its ability to create content, from text to images. This technology is democratizing creativity, allowing anyone to leverage algorithms that can mimic human creativity, business (https://www.coursera.org/articles/ai-trends).

Multimodal AI: Enhancing Human-Like Interactions

As AI systems evolve, they are increasingly integrating multiple modalities, enhancing the capability of AI to understand and generate intuitive and effective in fields like customer service and virtual assistants.

Tailored AI Solutions: Customization at Its Best



The code is available here:
<https://www.dailydoseofds.com/p/hands-on-building-a-multi-agent-news-generator/>